

Understanding Host Network Stack Overheads

A single-laptop reproduction of Cai, Chaudhary, Vuppalapati, Hwang and Agarwal, ACM SIGCOMM 2021

Team ID	T022
Project ID	15
Team members	Kunal Singh Rathore (Roll No. 23110181) Gurudayal Meena (Roll No. 23110125)
Instructor	Sameer Gundurao Kulkarni
Date	11 September 2026

Abstract

Network links in datacenters have become about ten times faster in a few years, but CPU cores have not. The Linux network stack runs on those cores, so at 100 Gbps the host CPU, and not the network, limits throughput. The SIGCOMM 2021 paper *Understanding Host Network Stack Overheads* measures this on two servers joined by a 100 Gbps link: how many CPU cycles the kernel spends per byte, where they go, and how the picture changes with kernel optimisations, traffic patterns, packet loss, message sizes and congestion control. In this project we repeat the paper's experiments on one laptop. A laptop network of about 1 Gbps would be the bottleneck instead of the CPU, so we build the two servers as two network namespaces inside one Ubuntu virtual machine with four virtual CPUs and join them with a veth pair, a virtual cable with no line rate. Each experiment is a script that creates the hosts, applies one setting, pins every program to a CPU, records the load of every CPU each second, repeats each run three times in random order and computes the paper's metric, throughput per CPU core. We reproduced eight of the paper's ten experiment sections (34 conditions, 102 runs) and explain why the other two (DDIO and IOMMU) need server hardware. Our results agree with the paper in direction: segmentation and receive offloads give 4.5× more work per busy core; flows that share cores lose efficiency; 1.5 % packet loss costs 27 % of throughput; small messages cost far more CPU per byte than large ones; and a long flow that shares a core with short flows suffers most. Two results differ from the paper, and we show why: in a virtual machine the receiver's packet processing runs on the sender's CPU, so our sender core is the bottleneck where the paper's receiver is, and the pacing cost of BBR therefore shows up as 33 % lower throughput instead of being hidden. We also corrected our own first version of the code, which did not compute the paper's metric and had measurement errors.

Contents

1. Introduction	2
2. Background: the paper.....	3
3. Design: two hosts inside one laptop	5
4. Why one laptop and not two	7
5. What a virtual machine cannot reproduce	8
6. How we measure	9
7. Experiments and results.....	10
8. Discussion: our results against the paper.....	21
9. Implementation, and what we fixed in our first version	24
10. Non-functional evaluation.....	25
11. Challenges faced	26
12. Conclusion and future work.....	27
References.....	29
Appendix A. How to run everything.....	30
Appendix B. Repository layout and data	30
Appendix C. Environment record	31
Appendix D. Glossary	32

1. Introduction

1.1 Motivation

For a long time the network was the slow part of a computer system, and the software that sends and receives data, the host network stack, was fast enough by comparison. In datacenters this is no longer true. Link speeds went from 10 Gbps to 100 Gbps within a few years, while the speed of one CPU core stayed almost the same. The network stack is ordinary software running on those cores: every byte an application sends is copied into the kernel, split into packets, given headers, queued and handed to the network card, and every byte received goes through the same steps in reverse. At 100 Gbps this work can take more than one full core for a single flow. So the CPU of the host, not the network, limits throughput. To design faster stacks, smarter network cards or better schedulers one first has to know where those CPU cycles go. That is what the paper we chose measures.

1.2 The paper

Cai, Chaudhary, Vuppapapati, Hwang and Agarwal, *Understanding Host Network Stack Overheads*, ACM SIGCOMM 2021 [1], is a measurement study of the Linux network stack at 100 Gbps. The authors connect two servers with a 100 Gbps cable, measure **throughput per core** (gigabits per second delivered per CPU core that was busy) and use the `perf` profiler to split the CPU time into eight categories: data copy, TCP/IP processing, network-device work, packet-descriptor management, memory allocation, locking, scheduling and other. They repeat this for the kernel's main optimisations (segmentation offload, receive offload, jumbo frames, flow steering, direct cache access), for five traffic patterns (single flow, one-to-one, incast, outcast, all-to-all), for random packet loss, for short request/ response flows and mixed workloads, for the IOMMU and for three congestion-control algorithms. Their main findings are that one core moves only about 42 Gbps even with every optimisation on, that copying data between kernel and application memory is the largest cost, and that sharing cores and caches between flows hurts efficiency. Section 2 goes through the paper in detail.

1.3 Objectives

1. Reproduce the paper's experiments on the hardware we have, one laptop, using the same tools (iperf, netperf, sysstat, perf) and the same settings (3 s warm-up, CPU pinning, the same offload and buffer configurations).
2. Measure the paper's metric, throughput per CPU core, from the load of every CPU during the measured window, instead of only reporting throughput.
3. Show which kernel features and traffic patterns cost CPU, and explain each result by the mechanism behind it, including the places where our setup necessarily differs from the paper's.
4. Turn our first, partly manual version of the project into scripts that create their own hosts, record every setting, check their own measurements and can be re-run with one command.

1.4 What we did

- We built the paper's two servers as two network namespaces inside one Ubuntu 20.04 virtual machine with 4 vCPUs, joined by a veth pair. CPUs 0 and 1 belong to the sender host, CPUs 2 and 3 to the receiver host (Section 3). Section 4 explains why one laptop is the right choice and why two laptops would only measure the cable between them.
- We wrote one experiment script per paper section, a shared module, an analysis script that turns the raw runs into graphs, tables and a summary file, and the scripts that build this report and the slides from that summary (Section 9, Appendix B). In total we ran 34 experimental conditions, each three times, which is 102 measured runs.
- We reproduced eight of the paper's ten experiment sections: single flow with the optimisations switched on one at a time and with different TCP buffer sizes (§3.1), the five traffic patterns and flow-count scaling (§3.2–3.5), random packet loss (§3.6), RPC sizes and mixed long/short flows (§3.7) and congestion control (§3.10). We also implemented the paper's CPU-breakdown method (Table 1). Sections 3.8 (DDIO) and 3.9 (IOMMU) cannot be done in a virtual machine without a physical network card; Section 5 explains why.
- We found and corrected the mistakes in our first version (Section 9): the paper's metric was missing, CPU time came from iperf3's own process, loss counters accumulated across runs, setups were made by hand and not recorded, and partial runs were mixed with good ones.
- We evaluated the measurement system itself: repeatability, background load, and the checks that each setting was really in effect (Section 10).

1.5 Main findings

- Segmentation and receive offloads (TSO/GSO/GRO) matter most: one flow goes from 18.4 to 108 Gbps on the same CPU, which is 4.5× more work per busy core.
- Sharing cores costs efficiency. Two independent core pairs give 1.57× the throughput of one flow, not 2×. Two flows into one receiver core are limited by that core, and eight flows into one receiver core deliver −5 % per busy receiver core compared with one flow.
- Random loss of 1.5 % costs 27 % of throughput. The measured loss rate (1.502 %) matches the configured one, which shows the loss emulation works.
- Small messages are expensive: 4 KB requests move 9.2 Gbps per busy receiver core, 64 KB requests 67.5 Gbps. A long flow that shares a core with 16 short flows loses 90 % of its throughput.
- Two results differ from the paper, and both have the same cause, which we measured directly: with a veth the receiver's packet processing runs on the sender's CPU (64 % of that core is softirq time when offloads are off). So our sender core is the bottleneck, whereas in the paper the receiver is. Because of this, BBR, whose pacing costs CPU at the sender, is 33 % slower than CUBIC in our setup, while the paper found the three algorithms equal.

1.6 How this report is organised

Section 2 explains the paper: its question, testbed, metric, the Linux data path and its findings. Section 3 describes our design and maps each element of the paper's testbed to ours. Section 4 explains why we used one laptop. Section 5 lists what a virtual machine cannot reproduce. Section 6 describes how we measure. Section 7 presents each experiment: what the paper did, what we did (with the exact commands), the graph and table, what we see, why, and how it compares with the paper. Section 8 discusses agreements and differences. Section 9 covers the implementation and the errors we fixed. Section 10 evaluates the non-functional qualities, Section 11 lists the challenges and Section 12 concludes. The appendices contain the commands to re-run everything, the repository layout, the environment record and a glossary. A separate supplement in the same folder repeats every experiment with its commands and answers likely questions.

2. Background: the paper

2.1 The question

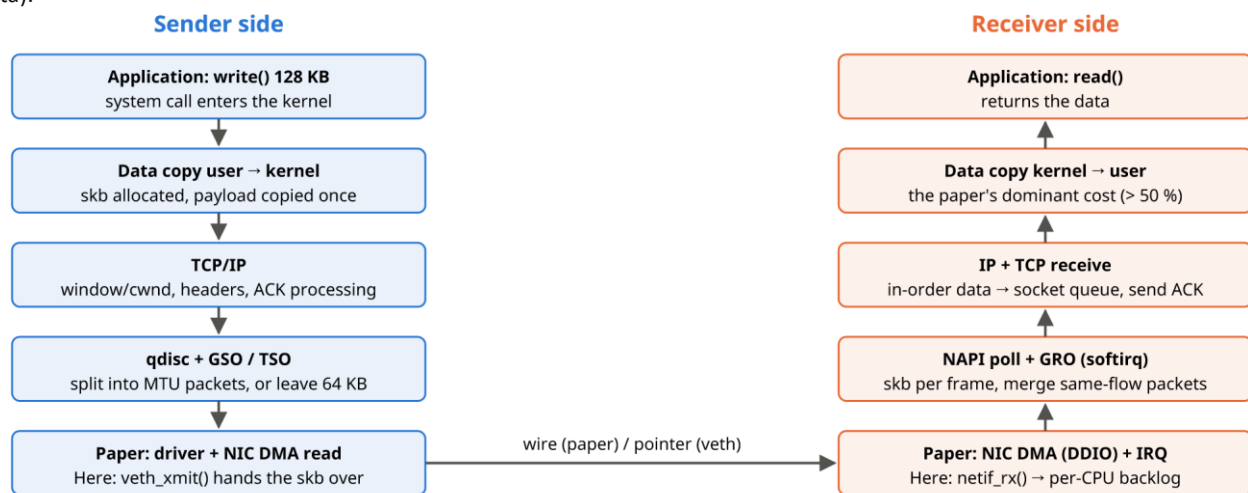
Datacenter network links became 4–10× faster in a few years (10/40 Gbps → 100 Gbps and more), but CPU cores did not get much faster and caches did not get much bigger (the slowdown of Moore's law and the end of Dennard scaling). The network stack is software that runs on those cores, so the old assumption "the network is the bottleneck, the host is fast enough" stopped being true. The paper measures **how many CPU cycles the Linux kernel spends per byte it sends or receives at 100 Gbps, where those cycles go, and how this changes with traffic patterns, flow sizes, packet loss and kernel features**. Earlier studies looked mostly at short flows on slow links, where TCP/IP protocol processing was the main cost; this paper shows that at high speed the main cost shifts to **copying data** between kernel and application memory.

2.2 Their testbed

Item	Paper (Section 2.2)
Machines	Two servers connected directly by one 100 Gbps cable, no switch (so the network can never be the bottleneck)
CPU	4-socket Intel Xeon Gold 6128, 6 cores per socket at 3.4 GHz (24 cores, 4 NUMA nodes); L1/L2/L3 = 32 KB / 1 MB / 20 MB; hyper-threading off
NIC	Mellanox ConnectX-5 Ex 100 Gbps, attached to one socket (the "NIC-local" NUMA node)
Software	Ubuntu 16.04, Linux 5.4.43; iperf for long flows, netperf for RPCs; sysstat for CPU use; perf for CPU profiles
Defaults	DDIO on, IOMMU off, TCP CUBIC, all optimisations on (TSO, GRO, jumbo frames 9000 B, aRFS) unless the experiment changes them

2.3 How a byte travels through Linux (the paper's Figure 1)

The figure below follows one write from the sending application to the receiving application. There is exactly **one data copy on each side**; everything else in the kernel works on metadata (the `skb`, "socket buffer", describes a packet or a group of packets and points to the data).



In the VM, the two lowest receiver steps run on the CPU that called `veth_xmit()`, i.e. usually the SENDER's core, unless RFS steers them to the receiver's core.

Sender. (1) The application calls `write()`. (2) The kernel allocates skbs and copies the data from the application's buffer into kernel memory. (3) TCP decides how much it may send (congestion window, receiver window) and builds headers. (4) The queueing discipline (qdisc) queues the packet; GSO splits large skbs into MTU-sized packets in software, or TSO lets the NIC split them in hardware. (5) The driver gives the NIC the addresses and the NIC reads the data by DMA. Almost all of this runs on the application's own core.

Receiver. (1) The NIC writes each arriving frame into memory by DMA (with Intel DDIO directly into the L3 cache). (2) It raises an interrupt on a core chosen by a steering mechanism (RSS, or aRFS: "the core where the application runs"). (3) NAPI polls the receive ring and builds one skb per frame. (4) GRO merges consecutive packets of the same flow into one large skb, so the layers above run once per 64 KB instead of once per 1500 B. (5) IP and TCP process the skb and append in-order data to the socket's receive queue; TCP sends ACKs. (6) The application calls `read()` and the kernel copies the data into the application's buffer. With aRFS all six steps run on one core.

2.4 How they measure

Throughput-per-core = total throughput ÷ CPU utilisation (in cores) of the bottleneck host. Example: 42 Gbps while the receiver uses 100 % of one core = 42 Gbps per core. The metric is used instead of plain throughput because at 100 Gbps one flow cannot even fill the link: what matters is *how many cores* a given rate costs. CPU utilisation is measured with `sysstat` (all kernel and application work on every core). To see *where* the time goes they sample with `perf`, take the functions covering about 95 % of CPU time, and sort them into eight categories (Table 1 of the paper):

Category	What it contains
data copy	copying payload user → kernel (sender) and kernel → user (receiver)
tcp/ip processing	all TCP and IP protocol work
netdevice subsystem	driver and device layer: NAPI polling, GSO/GRO, qdisc
skb management	building, splitting and releasing skbs
memory alloc/dealloc	allocating and freeing skbs and pages
lock/unlock	spin locks and socket locks
scheduling	context switches, waking and sleeping threads
etc.	everything else, e.g. interrupt handling, system-call entry

2.5 The optimisations the paper switches on and off

Name	What it does	Why it saves CPU
------	--------------	------------------

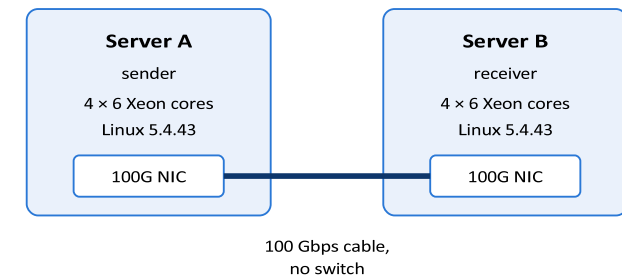
TSO / GSO	TCP Segmentation Offload / Generic Segmentation Offload: TCP builds one 64 KB skb; the NIC (TSO) or the kernel just before the driver (GSO) cuts it into MTU packets	TCP/IP runs once per 64 KB instead of once per 1500 B
GRO / LRO	Generic / Large Receive Offload: the receiver merges packets of one flow into one large skb (GRO in software, LRO in the NIC)	the same saving on the receive side
Jumbo frames	MTU 9000 instead of 1500	6× fewer packets and skbs; fewer packets for GRO to merge
RSS / RPS / RFS / aRFS	choose the core that processes an incoming packet: by a hash (RSS in the NIC, RPS in software) or the core where the receiving application runs (RFS in software, aRFS in the NIC)	with aRFS, interrupt, TCP/IP and application share one core's cache, and the socket is not bounced between cores
DCA / DDIO	Direct Cache Access: the NIC writes packets straight into the L3 cache instead of DRAM	the receiver copies from cache, not from memory

2.6 What the paper found

Section	Finding
3.1 Single flow	With every optimisation on, one core processes about 42 Gbps (55 Gbps with hand-tuned buffers). The receiver is the bottleneck. Data copy is the largest cost (about 50 % of receiver cycles). Even one flow sees a 49 % L3 miss rate: with large TCP buffers, data written into the cache by DDIO is evicted before the application copies it. A core on the NIC-remote NUMA node loses about 20 %.
3.2 One-to-one	The link saturates at 8 flows; throughput-per-core falls by 64 % at 24 flows (less cache locality, less GRO merging, more scheduling as cores idle).
3.3 Incast	Many flows into one receiver core: throughput-per-core falls about 19 % at 8 flows because the flows fight over the cache (miss rate 48 % → 78 %). TCP is sender-driven, so the receiver cannot limit how many flows hit one core.
3.4 Outcast	One sender core to many receivers reaches 89 Gbps per sender core : the sender pipeline is about 2× more efficient than the receiver's (TSO is in hardware; the sender's cache is warm).
3.5 All-to-all	Up to 24×24 flows: throughput-per-core falls about 67 %; with many flows each flow gets few packets per interval, so GRO cannot merge and skbs get smaller.
3.6 Packet loss	Drop rate 0 → 1.5 %: throughput-per-core falls about 24 %; more ACK and retransmission work, higher at the sender.
3.7 Flow sizes	Small RPCs (4 KB) are dominated by per-message costs (TCP/IP, scheduling), not copying; from 16 KB data copy dominates again. Mixing one long flow with 16 short flows on a core cuts throughput-per-core about 43 % (long flow –48 %, short flows –42 %).
3.8 DCA	Turning DDIO off costs 19 %.
3.9 IOMMU	Turning the IOMMU on costs 26 % (page mapping/unmapping; memory management reaches 30 % of receiver cycles).
3.10 Congestion control	CUBIC, BBR and DCTCP give almost the same throughput-per-core, because the receiver is the bottleneck and all three differ only at the sender. BBR has more sender scheduling overhead because of pacing.

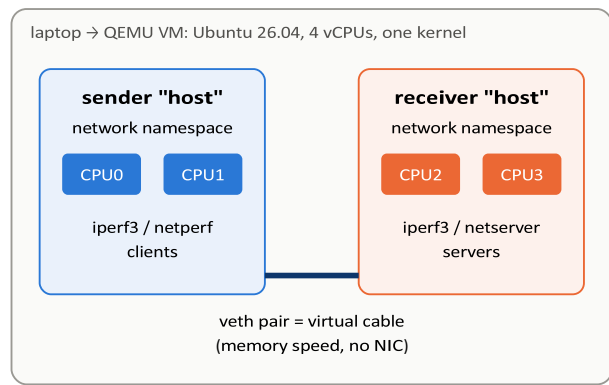
Their overall message: host resources (cores, caches) must be orchestrated like network resources; zero-copy receive, receiver-driven transports, and CPU schedulers that know about the network stack are the promising directions (Section 4 of the paper).

3. Design: two hosts inside one laptop



project: one laptop

two kernels, two sets of caches,
real DMA, interrupts, DDIO



The
paper:
two
servers
This

3.1 Machine

Item	This project
Physical machine	One Apple-silicon laptop (the VM reports CPU vendor "Apple")
Virtual machine	QEMU, Ubuntu 26.04 LTS, Linux 7.0.0-31-generic, aarch64
CPUs	4 vCPUs, 1 thread per core, a single NUMA node
Memory	7.2 GiB
Performance counters	None: only software perf events (no cache-miss counters)
Tools	iperf 3.20, netperf 2.7.1, sysstat (mpstat), perf 7.0, ethtool, iproute2 (ip, tc, ss, nstat)

3.2 How two "servers" are made

A **network namespace** is a separate copy of the kernel's network stack: its own interfaces, IP addresses, routing table, sockets and TCP settings. A program started "inside" a namespace sees only that namespace's network. A **veth pair** is two virtual Ethernet interfaces joined back-to-back: whatever is sent into one comes out of the other. Putting one end in each namespace gives two hosts joined by a cable. Each experiment script builds its own pair, for example:

```
ip netns add tp_tx ; ip netns add tp_rx                # two "hosts"
ip -n tp_tx link add tp_tx0 type veth peer name tp_rx0 netns tp_rx  # the "cable"
ip -n tp_tx address add 10.231.2.1/24 dev tp_tx0 ; ip -n tp_rx address add 10.231.2.2/24 dev tp_rx0 ip
netns exec tp_rx taskset -c 2 iperf3 -s -1 -B 10.231.2.2 -p 5301      # receiver on CPU 2 ip netns
exec tp_tx taskset -c 0 iperf3 -c 10.231.2.2 -p 5301 -C cubic -O 3 -t 15 -J  # sender on CPU 0
```

`taskset -c N` pins a program to CPU N, as the paper pins each application to one core. The paper's servers have 24 cores each; our VM has 4, so each "host" gets two: **CPUs 0 and 1 for the sender, CPUs 2 and 3 for the receiver**. The namespaces are deleted at the end of every experiment, so every run starts from the same clean state.

3.3 What corresponds to what

Paper	This project	Same?
Two servers	Two network namespaces in one kernel	Partly: separate network stacks, but shared CPUs, memory, caches and scheduler
100 Gbps cable, no switch	veth pair (the skb pointer is handed over; nothing is copied or serialised)	Same role: never the bottleneck
Application core per flow	<code>taskset</code> to one vCPU	Yes
TSO in the NIC, GSO	TSO/GSO on veth: the 64 KB skb crosses the veth unsplit	Same effect, done in software

GRO	GRO on the veth (uses NAPI inside veth)	Yes
Jumbo frames	MTU 9000 on both veth ends	Yes
aRFS	RFS (software Receive Flow Steering), only in the single-flow study; needs root	Same idea, done in software
Interrupts, NAPI on a NIC queue	veth delivers with <code>netif_rx()</code> into the current CPU's backlog, handled in <code>softirq</code>	No hardware interrupt
Switch dropping packets (§3.6)	netem on an IFB device at the receiver's input	Yes (random drop)
iperf / netperf	iperf3 (one process per flow) / netperf TCP_RR and TCP_STREAM	Yes
sysstat CPU utilisation	mpstat, per core, every second	Yes
perf profiles	perf with the <code>cpu-clock</code> timer event	Yes (timer-based sampling)
DDIO, IOMMU, NUMA, PMU counters	Not present	No (Section 5)

4. Why one laptop and not two

Short answer

The paper only makes sense when the **CPU is the bottleneck and the network is not**. Laptop networks (1 Gbps Ethernet, Wi-Fi) are 100× slower than the paper's link, so with two laptops the link would be full while the CPUs were almost idle, and every experiment would just measure the link speed. A veth pair inside one machine is a "link" that runs at memory speed, so the CPU becomes the bottleneck again: the same situation the paper creates with 100 Gbps hardware.

4.1 The paper deliberately removes the network as a bottleneck

Section 2.2 of the paper: "To ensure that bottlenecks are at the network stack, we setup a testbed with two servers directly connected via a 100Gbps link (without any intervening switches)." Every result in the paper (throughput-per-core, the CPU breakdowns, the effect of TSO/GRO, of traffic patterns, of congestion control) assumes that the application and kernel on the hosts are what limits the speed.

4.2 Two laptops would measure the cable, not the stack

In our single-flow runs the CPUs moved about **108.0 Gbps** and delivered **60.9 Gbps per busy core** (counting both hosts). At that efficiency, a 1 Gbps link would keep the stack busy for only about **1.6 % of one core**, and even a 10 Gbps adapter only about **16 %**. With two laptops:

- every experiment would report "throughput ≈ link speed" (about 0.94 Gbps of TCP payload on 1 Gbps Ethernet, and a varying 0.2–1 Gbps on Wi-Fi);
- switching TSO/GRO off, changing the traffic pattern or the congestion control would change only the CPU percentage, which would sit at a few percent, lost in background noise;
- the single-flow, one-to-one, incast, outcast and all-to-all patterns would all give the same throughput, because they would all share the same 1 Gbps link;
- Wi-Fi adds interference, rate changes and retransmissions that have nothing to do with the host stack; the paper even avoids a switch.

In short, the host overheads the paper studies would be invisible. To see them between two machines we would need 100 Gbps NICs and a direct cable, as in the paper.

4.3 Why a veth pair recreates the paper's situation

- A veth pair has no line rate. "Transmitting" hands the skb to the other end; the speed is limited only by how fast the CPUs can run the stack. That is exactly the condition the paper needs.

- Evidence that the CPU is the limit: throughput changes when only CPU work changes (for example TSO/GSO/GRO off vs on, Section 7.1), and one CPU core is close to 100 % busy in the single-flow runs.
- Both ends run the **real Linux TCP/IP stack** (the same code as between two servers): sockets, TCP CUBIC/BBR/DCTCP, qdiscs, GSO/GRO, softirq processing, data copies.
- Every experiment is controlled and repeatable: the scripts build identical fresh hosts every time, pin every program, and record every setting.

4.4 The price we pay

Without a physical NIC there is no DMA, no hardware interrupt and no DDIO, and both "hosts" share one kernel and one set of caches. Section 5 lists every consequence. The most visible one: the receiver's packet processing often runs on the *sender's* CPU, so in our VM the **sender core is usually the bottleneck**, while in the paper the receiver is. We measure and explain this in every experiment instead of hiding it.

4.5 If two laptops are still required

The scripts only need the two namespace names replaced by two machines' addresses. The expected result is "throughput = link speed" with low CPU use. That run could be added as a sanity check showing why the paper needs 100 Gbps; it cannot reproduce the paper's findings.

5. What a virtual machine cannot reproduce

Paper item	Why it is not possible here	What we do instead
§3.8 DCA / DDIO	DDIO is an Intel Xeon feature that lets a NIC write into the L3 cache. Our CPU is Apple silicon, and a veth has no NIC and no DMA at all.	Explained only. Our receiver copies data the sender core has just written, which is probably still in a shared cache, so it behaves like a "perfect DDIO".
§3.9 IOMMU	The IOMMU translates addresses used by a device's DMA. There is no device doing DMA in our data path.	Explained only.
NIC-remote NUMA (Fig. 4)	The VM has one NUMA node.	Not applicable.
L3 cache-miss rates (Figs. 3e, 4, 6c, 10c)	They need hardware performance counters (PMU). This VM exposes none: <code>/sys/bus/event_source/</code> devices lists only software, tracepoint, kprobe, uprobe and breakpoint.	We report throughput and CPU use only; cache explanations are marked as the paper's.
NIC ring size (Fig. 3e)	A veth has no hardware receive descriptors.	Only the TCP buffer half of Fig. 3(e) is repeated.
NAPI-to-copy latency (Fig. 3f)	Needs custom timestamps inside the kernel.	Not done.
aRFS	A hardware NIC feature.	RFS (the software version) as the fourth single-flow level. It needs a global kernel setting, so it runs only under real sudo.
1–24 flows per pattern	4 vCPUs in total.	2 CPUs per host; incast and outcast scaled to 8 flows on one core.
Separate kernels and caches	Both namespaces share one kernel, one page allocator, one scheduler and the same caches.	Accepted; per-CPU numbers show who does what.

Quiet dedicated servers	The VM also runs the desktop and editor, and its vCPUs are threads on macOS.	Idle CPU load is measured before every experiment and reported.
-------------------------	--	---

The most important difference: where receive processing runs

With a real NIC and aRFS, the receiver's interrupt, NAPI, GRO and TCP/IP processing run on the receiver's core. With a veth, `veth_xmit()` passes the skb to `netif_rx()`, which queues it on the backlog of the **CPU that is currently running**, and the NET_RX softirq processes it there. For data packets that is usually the sender's core; for ACKs it is the receiver's core. So in our VM:

- the sender core does its own work *plus* much of the receiver's IP/TCP receive work (visible as "softirq" time on CPU0 in the graphs);
- the receiver core mostly does the kernel → user data copy and ACK work;
- therefore the **sender core is usually the busiest**, the opposite of the paper.

RFS changes this: it steers each received packet to the core where the receiving application last ran, which is what aRFS does in the paper. The fourth single-flow level measures that effect.

6. How we measure

6.1 Run protocol (the same for every experiment)

- Fresh namespaces and veth per experiment; settings recorded in `configuration.json` (offloads from `ethtool -k`, qdiscs, MTU, TCP settings, versions).
- One iperf3 client/server process pair **per flow**, each pinned with taskset; all flows of a run start together.
- `iperf3 -O 3 -t 15`: the first 3 s (TCP slow start) are omitted, the next 15 s are measured, as in the original scripts. netperf runs 20 s and the first 2 s are excluded from the CPU window.
- 3 repetitions per condition, in a **random order fixed by seed 42**, so slow changes in background load do not favour whichever condition runs first. 2 s pause between runs.
- A run counts only if every process exits successfully and its output parses; a folder gets `COMPLETED.txt` only when all its runs succeeded, and the analysis ignores folders without it.

6.2 How CPU use and throughput-per-core are computed

`mpstat -P ALL 1` prints, every second, how each CPU spent that second: user, system (kernel on behalf of a process), softirq/irq (packet processing outside any process) and idle. For each run we average the per-second lines that fall inside the measured 15 s window (its start and end come from iperf3's own timestamps). Then:

```

busy(cpu)          = 100 % - idle %          (all work: user + kernel + softirq + irq)
sender_cores       = Σ busy / 100 over the sender CPUs the run used (e.g. CPU0)
receiver_cores     = Σ busy / 100 over the receiver CPUs the run used (e.g. CPU2)
throughput per busy core (both hosts) = throughput / (sender_cores + receiver_cores)
throughput per core (bottleneck side) = throughput / cores of the side whose busiest CPU is most loaded

```

The second formula is the paper's definition. We also give the first because in one VM part of the receiver's work runs on the sender's CPUs (Section 5), so neither side on its own is a clean "host". Example: 100 Gbps with CPU0 at 99 % and CPU2 at 76 % gives $100 / 1.75 = 57$ Gbps per busy core for both hosts, and $100 / 0.99 = 101$ Gbps per core for the bottleneck (sender) side.

Why not iperf3's own CPU number? iperf3 reports only the CPU time of its own process (user + system). It misses softirq packet processing, which runs outside the process and, in a veth, often on the other host's CPU. The paper uses sysstat for exactly this reason.

6.3 Checks that make the numbers trustworthy

- Offload settings are read back with `ethtool -k` after every change and the run aborts if they differ.
- Congestion control: the algorithm is read from the live data socket with `ss -ti` (found by its fixed source port) twice per run. For DCTCP the script also checks that ECN was negotiated.
- Packet loss: the netem queue is re-created before every run and its packet and drop counters give the *measured* loss rate, which is compared with the configured one.

- Throughput is the receiver-side number (`sum_received`), i.e. data that actually arrived.

6.4 Background load and statistics

Background load measured with the experiment idle (5 s just before it started; busy % per CPU):

Experiment	CPU0	CPU1	CPU2	CPU3
Single flow	11 %	11 %	10 %	11 %
Traffic patterns	1 %	2 %	2 %	1 %
Packet loss	4 %	8 %	4 %	5 %
Flow sizes	10 %	10 %	12 %	13 %

Numbers are the mean $\pm$ sample standard deviation (SD) of 3 runs. With 3 runs, treat a difference smaller than about two SDs as "no clear difference". The VM shares the laptop with other programs, so run-to-run variation of 5–10 % is normal here.

The per-CPU logs make disturbances visible: a CPU that the experiment does not use should be nearly idle. During our first full run, a second editor window started in the VM and pushed the unused CPUs from under 10 % to as much as 52 % busy, and throughput dropped at the same moment. Those two experiments (single flow and traffic patterns) were therefore discarded and repeated with the VM quiet; the results in this report come from the repeated runs.

7. Experiments and results

Each subsection has the same structure: what the paper did and found; a box with the question, setup, steps and measurements of our experiment; the graph and table; what we see; why; and how it compares with the paper.

The experiments at a glance

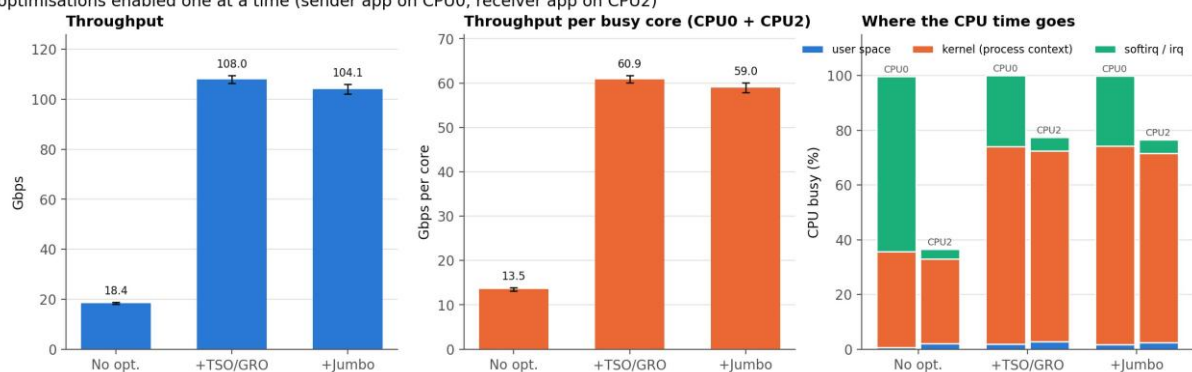
	Experiment	Paper	Main question	Runs
7.1	Single flow, optimisations	§3.1, Fig. 3a/b	What does one flow cost; what does each optimisation save?	4 levels $\times$ 3
7.2	TCP buffer size	§3.1, Fig. 3e	Does buffer size change speed/efficiency?	6 sizes $\times$ 3
7.3	Five traffic patterns	§3.2–3.5, Fig. 2	What does sharing cores between flows cost?	5 patterns $\times$ 3
7.4	More flows per core	Figs. 6 and 7	How does efficiency change with 1–8 flows on one core?	2 $\times$ 2 extra $\times$ 3
7.5	Packet loss	§3.6, Fig. 9	What does random loss cost?	4 rates $\times$ 3
7.6	Short and mixed flows	§3.7, Figs. 10/11	Small RPCs vs long flows; mixing both on a core	9 cases $\times$ 3
7.7	Congestion control	§3.10, Fig. 13	CUBIC vs BBR vs DCTCP	3 $\times$ 3
7.8	CPU breakdown	Table 1, Fig. 3c/d	Where do the cycles go?	4 levels
7.9	DCA / DDIO	§3.8, Fig. 12	What does cache DMA save?	not possible here
7.10	IOMMU	§3.9, Fig. 12	What does address translation cost?	not possible here

7.1 Single flow, optimisations one at a time (paper §3.1, Fig. 3a/3b)

Paper. One long flow between two NIC-local cores. Starting from "no optimisations" they add TSO/GRO, then jumbo frames, then aRFS. Each step raises throughput-per-core, reaching about 42 Gbps; the receiver core is always the bottleneck.

In the paper	Two servers, 100 Gbps cable. One iperf flow between NIC-local cores. Optimisations switched on one at a time: none → TSO/GRO → jumbo frames (MTU 9000) → aRFS. Without aRFS the interrupts are pinned to a core on another NUMA node (worst case).
Question	How much CPU does one long TCP flow cost, and how much does each kernel optimisation save?
Setup	One iperf3 flow: sender application pinned to CPU0, receiver application pinned to CPU2; CUBIC; 128 KB writes. Script <code>exp_single_flow.py</code> .
Steps	1. Create the two namespaces and the veth. 2. Set the level: No opt. = <code>ethtool -K ... tso off gso off gro off</code> , MTU 1500 → +TSO/GSO/GRO = all three on → +Jumbo = MTU 9000 → +RFS = receive processing steered to the receiver's core (sudo only). 3. Run iperf3: 3 s warm-up discarded, 15 s measured, mpstat logging every CPU each second. 4. Three runs per level, levels in random order.
Measured	Throughput; busy % of CPU0 and CPU2 split into user / kernel / softirq; throughput per busy core; retransmissions.
Reading the figure	Left: speed. Middle: efficiency (Gbps per busy core). Right: what each core spent its time on; softirq on CPU0 is receive work done on the sender's core.
Run it	<code>sudo python3 code/exp_single_flow.py</code> (all four levels and the buffer sizes, ≈ 12 min). Options: <code>--levels no-opt,tso-gro,jumbo</code> (without RFS), <code>--no-buffers</code> , <code>--reps 5</code> .
Commands inside	<pre> ip netns add sf_tx ; ip netns add sf_rx ip -n sf_tx link add sf_tx0 type veth peer name sf_rx0 netns sf_rx ip netns exec sf_tx ethtool -K sf_tx0 tso off gso off gro off # No opt. (same on sf_rx0) ip netns exec sf_tx ethtool -K sf_tx0 tso on gso on gro on # +TSO/GSO/GRO ip -n sf_tx link set sf_tx0 mtu 9000 # +Jumbo (both ends) echo 32768 > /proc/sys/net/core/rps_sock_flow_entries # +RFS, plus in each namespace: echo 32768 > /sys/class/net/sf_rx0/queues/rx-0/rps_flow_cnt ip netns exec sf_rx taskset -c 2 iperf3 -s -l -B 10.231.1.2 -p 5301 ip netns exec sf_tx taskset -c 0 iperf3 -c 10.231.1.2 -B 10.231.1.1 -p 5301 -P 1 -C cubic -O 3 -t 15 -J --get-server-output mpstat -P ALL 1 # per-CPU log during the run </pre>

Single flow, optimisations enabled one at a time (sender app on CPU0, receiver app on CPU2)



Single flow. Left: throughput. Middle: throughput per busy core counting CPU0 + CPU2. Right: how CPU0 (sender app) and CPU2 (receiver app) spent their time. Mean ± SD of the runs.

Level	Throughput (Gbps)	Gbps per busy core (both hosts)	Gbps per core (bottleneck side)	CPU0 busy (softirq)	CPU2 busy (softirq)	Retrans.
No opt.	18.4 ± 0.4	13.5 ± 0.4	18.5 ± 0.3	100 % (64 %)	37 % (3 %)	1,487
+TSO/GSO/GRO	108.0 ± 1.5	60.9 ± 0.8	108.0 ± 1.4	100 % (26 %)	77 % (5 %)	5
+Jumbo (MTU 9000)	104.1 ± 2.0	59.0 ± 1.1	104.2 ± 2.0	100 % (26 %)	77 % (5 %)	40

What we see. Switching on TSO/GSO/GRO is by far the biggest step: throughput goes from 18.4 to 108.0 Gbps and throughput per busy core from 13.5 to 60.9 (4.5×). Jumbo frames then change throughput per busy core by −3 %.

Why. With no offloads every skb holds one 1500-byte packet, so all per-packet work (TCP/IP, qdisc, the veth hand-over, the receive path) is repeated about 1.5 million times per second. With TSO/GSO the sender gives veth 64 KB skbs and veth passes them through whole, so per-packet costs are paid about 44× less often, which is the same reason TSO/GRO help in the paper. Jumbo frames matter much less than in the paper: there, GRO must merge 1500-byte frames arriving from the NIC, and jumbo frames give it 6× fewer frames to merge. In veth nothing is split in the first place, so the MTU barely matters once TSO is on. This is expected.

Who is the bottleneck. In the "No opt." case CPU0 spends 64 % of its time in softirq, while CPU2 is only 37 % busy. That softirq time is the receiver's IP/TCP processing running on the sender's core (Section 5). With offloads on, CPU0 is still the busier core (100 % vs 77 %). **So the sender side is the bottleneck here, while in the paper it is the receiver.** Two reasons: (1) veth makes the sender's core do receive work; (2) the paper's receiver copies data that the NIC wrote into memory (often a cache miss), while ours copies data the sender core wrote a moment earlier.

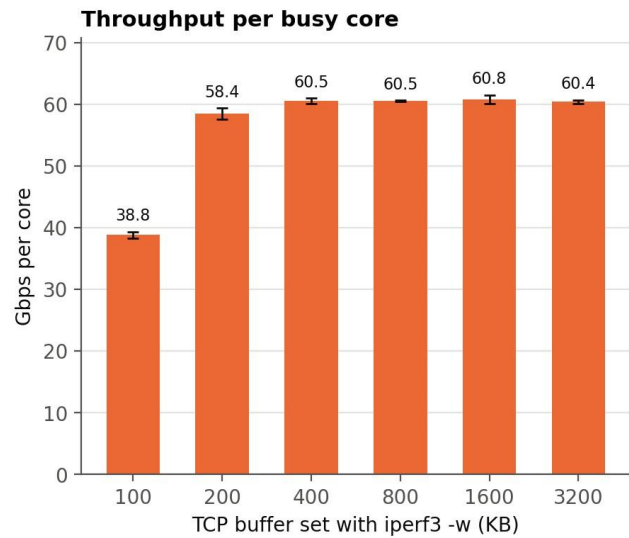
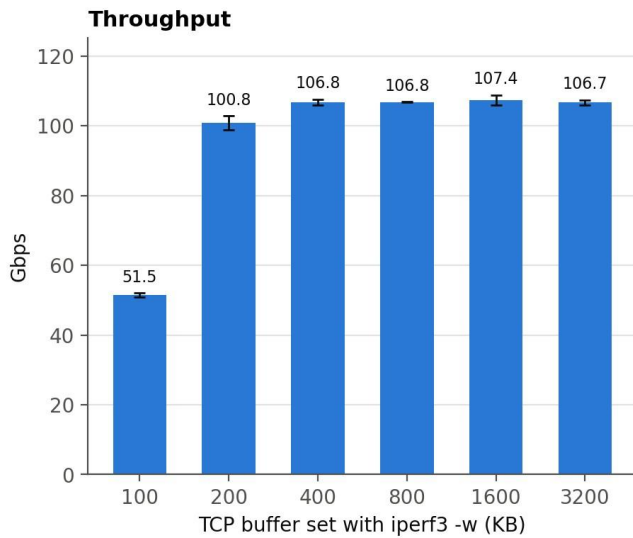
The RFS level needs real root (a global kernel table), so it is absent from runs made without sudo. `sudo python3 code/exp_single_flow.py` adds it.

Compared with the paper. Same direction and the same ranking of the offloads: segmentation/ receive offload is the key optimisation. Our absolute numbers are higher than 42 Gbps per core because there is no NIC, DMA or interrupt cost, the data stays in cache, and the receive work is split over two cores.

7.2 TCP buffer size (paper §3.1, Fig. 3e)

Paper. With the TCP receive buffer fixed (auto-tuning off), throughput-per-core *falls* for large buffers: TCP keeps a buffer's worth of data in flight, host latency grows, and data written into the L3 cache by DDIO is evicted before the application copies it (3200 KB and ≤ 512 descriptors gave their best result, about 55 Gbps).

In the paper	The same single flow with all optimisations. TCP receive buffer fixed between 100 KB and 12.8 MB (auto-tuning overridden) and NIC receive descriptors 128–8192; L3 cache miss rate measured with hardware counters.
Question	Does the size of the TCP buffer change speed and CPU efficiency?
Setup	Same flow as 7.1 at the +Jumbo level. <code>iperf3 -w SIZE</code> fixes the send and receive socket buffers and so turns Linux auto-tuning off, as the paper does. Above 4 MB the kernel caps the value (<code>net.core.rmem_max</code>), so we stop at 3200 KB, the paper's first six sizes.
Steps	Sizes 100, 200, 400, 800, 1600, 3200 KB; three runs each in random order; 3 s warm-up + 15 s.
Measured	Throughput and throughput per busy core for each size.
Reading the figure	A low bar means the buffer limits the flow; flat bars mean the buffer no longer matters.
Run it	Part of <code>sudo python3 code/exp_single_flow.py</code> (runs after the levels; <code>--no-buffers</code> skips it).
Commands inside	<pre>ip -n sf_tx link set sf_tx0 mtu 9000 # +Jumbo level, both ends ip netns exec sf_tx taskset -c 0 iperf3 -c 10.231.1.2 -p 5301 -C cubic -O 3 -t 15 -J -w 800K # -w = 100K, 200K, 400K, 800K, 1600K, 3200K (fixes both socket buffers)</pre>



Throughput and throughput per busy core against the fixed TCP buffer size.

Buffer	Throughput (Gbps)	Gbps per busy core	CPU0 busy	CPU2 busy
100 KB	51.5 ± 0.6	38.8 ± 0.5	69 %	63 %
Buffer	Throughput (Gbps)	Gbps per busy core	CPU0 busy	CPU2 busy
200 KB	100.8 ± 2.0	58.4 ± 0.9	96 %	76 %
400 KB	106.8 ± 0.9	60.5 ± 0.5	100 %	77 %
800 KB	106.8 ± 0.1	60.5 ± 0.1	100 %	77 %
1600 KB	107.4 ± 1.4	60.8 ± 0.7	100 %	77 %
3200 KB	106.7 ± 0.7	60.4 ± 0.3	100 %	77 %

What we see. With 100 KB the flow reaches only 51.5 Gbps; throughput per busy core then stays flat up to the largest size.

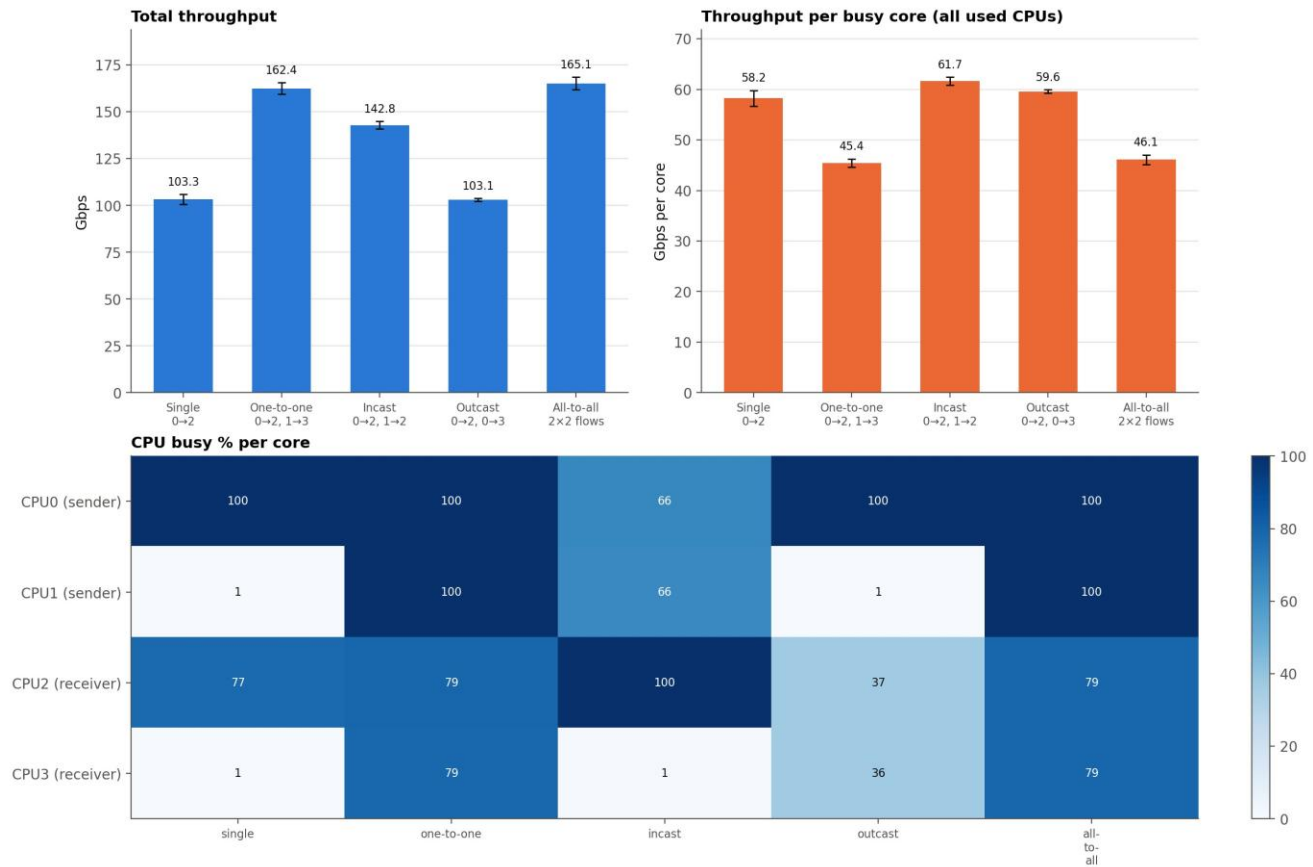
Why. At most one buffer of unacknowledged data can be in flight, so $\text{throughput} \leq \text{buffer} \div \text{round-trip time}$. A small buffer therefore limits the flow even though the CPU is not full: the cores wait. Once the buffer covers the bandwidth-delay product, a larger buffer adds nothing. The paper's decline for large buffers comes from DDIO cache eviction; our VM has no NIC DMA and no DDIO, so that mechanism cannot appear. This is a difference in hardware, not an error.

7.3 The five traffic patterns (paper §3.2–3.5, Fig. 2)

Paper. Single flow, one-to-one (each sender core to its own receiver core), incast (many sender cores into one receiver core), outcast (one sender core to many receiver cores) and all-to-all (every sender core to every receiver core). Sharing host resources hurts: throughput-per-core differs by up to 66 % between patterns.

In the paper	1 to 24 cores per server, all optimisations on. One-to-one: sender core $i \rightarrow$ receiver core i . Incast: n sender cores $\rightarrow$ 1 receiver core. Outcast: 1 sender core $\rightarrow n$ receiver cores. All-to-all: every sender core to every receiver core (up to $24 \times 24 = 576$ flows).
Question	What happens to efficiency when flows share sender or receiver cores?
Setup	Each host has two cores. Single: $0 \rightarrow 2$. One-to-one: $0 \rightarrow 2$, $1 \rightarrow 3$. Incast: $0 \rightarrow 2$, $1 \rightarrow 2$ (receiver core shared). Outcast: $0 \rightarrow 2$, $0 \rightarrow 3$ (sender core shared). All-to-all: $0 \rightarrow 2$, $0 \rightarrow 3$, $1 \rightarrow 2$, $1 \rightarrow 3$. Kernel default veth settings (TSO/GSO on, GRO off, MTU 1500), CUBIC. Script <code>exp_traffic_patterns.py</code> .
Steps	For every flow one iperf3 server pinned to its receiver core and one client pinned to its sender core, all started together; 3 s warm-up + 15 s. All patterns interleaved in one random order, three runs each.
Measured	Total throughput (sum of the flows), slowest and fastest flow (fairness), busy % of all four CPUs, throughput per busy core.

Reading the figure	The heat map shows which core is full (dark) in each pattern: that core limits the pattern.
Run it	<code>sudo python3 code/exp_traffic_patterns.py</code> (all 9 patterns, $\approx$ 11 min). Subset: <code>-patterns single,one-to-one,incast,outcast,all-to-all</code> ; more runs: <code>--reps 5</code> .
Commands inside	<pre># example: incast = two flows, sender CPUs 0 and 1, both receivers on CPU 2 ip netns exec tp_rx taskset -c 2 iperf3 -s -1 -B 10.231.2.2 -p 5301 & ip netns exec tp_rx taskset -c 2 iperf3 -s -1 -B 10.231.2.2 -p 5302 & ip netns exec tp_tx taskset -c 0 iperf3 -c 10.231.2.2 -p 5301 -C cubic -O 3 -t 15 -J & ip netns exec tp_tx taskset -c 1 iperf3 -c 10.231.2.2 -p 5302 -C cubic -O 3 -t 15 -J & # one-to-one: second pair uses receiver CPU 3; outcast: both clients on CPU 0</pre>



Top left: total throughput (sum of all flows). Top right: throughput per busy core over the CPUs each pattern uses. Bottom: average busy % of every CPU during the measured window (a CPU the pattern does not use shows only background load).

Pattern	Total Gbps	Gbps per busy core	Gbps per core, bottleneck side	CPU0/1/2/3 busy %	Slowest–fastest flow	Retrans.
Single (0→2)	103.3 ± 2.6	58.2 ± 1.5	103.4 ± 2.6	100 / 1 / 77 / 1	103.3–103.3	2
One-to-one (0→2, 1→3)	162.4 ± 3.1	45.4 ± 0.8	81.4 ± 1.5	100 / 100 / 79 / 79	80.9–81.5	52
Incast (0→2, 1→2)	142.8 ± 1.9	61.7 ± 0.8	142.8 ± 1.9	66 / 66 / 100 / 1	71.1–71.7	147
Outcast (0→2, 0→3)	103.1 ± 0.8	59.6 ± 0.3	103.1 ± 0.8	100 / 1 / 37 / 36	51.5–51.6	281
All-to-all (2×2)	165.1 ± 3.3	46.1 ± 1.0	82.6 ± 1.7	100 / 100 / 79 / 79	41.0–41.5	10,706

What we see.

- **One-to-one** gives 1.57× the throughput of a single flow: a second, independent pair of cores adds capacity because the veth "link" is not the limit. But it is not 2×: each flow reaches only 81.2 Gbps instead of 103.3, although it has its own two cores, and throughput

per busy core falls from 58.2 to 45.4 (–22 %). This is the paper's one-to-one finding (Fig. 5a): even with one flow per core, efficiency drops as flows are added, because the flows share the cache and memory system. Our four vCPUs also share the laptop processor's caches and memory bandwidth, and they are threads of one hypervisor.

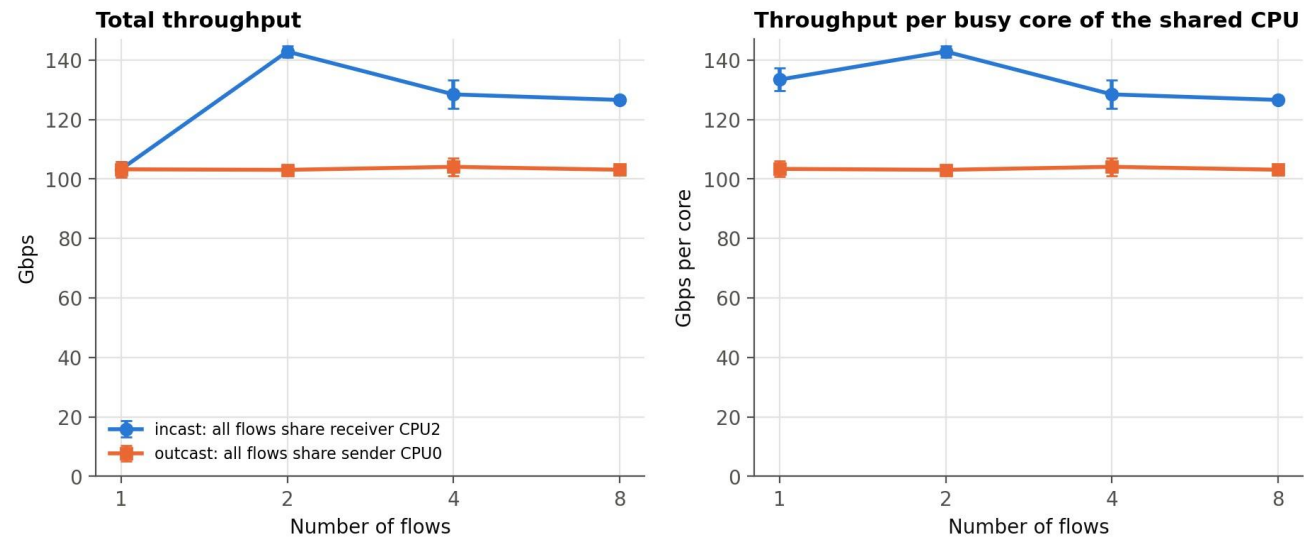
- **Incast** delivers –12 % vs one-to-one: both receiving applications share CPU2 (100 % busy), the paper's receiver contention.
- **Outcast** delivers –37 % vs one-to-one and about the same as one flow (103.1 vs 103.3 Gbps): one sender core (CPU0 100 % busy) must do the sending work for both flows plus their receive-side softirq work, so the shared sender core caps the total.
- **All-to-all** uses all four cores and reaches 165.1 Gbps (+2 % vs one-to-one); its throughput per busy core is 46.1 vs 45.4 for one-to-one.
- Across patterns, throughput per busy core ranges from 45.4 (one-to-one) to 61.7 (incast), a spread of 26 % (paper: up to 66 %).

Why. Whenever two flows share one core, that core must serve both, and its cache holds the data of both. The CPU heat map shows directly which core saturates in each pattern. In our VM the shared *sender* core (outcast) hurts most, because the sender core also carries receive work; in the paper the shared *receiver* core (incast) is the critical one.

7.4 More flows on one core (paper Figs. 6 and 7)

Paper. Incast with 1–24 flows into one receiver core: throughput-per-core falls about 19 % at 8 flows as the flows compete for the cache. Outcast with 1–24 flows from one sender core: throughput-per-sendercore *ris*es up to 8 flows (89 Gbps) and then stays about flat.

In the paper	Incast with 1, 8, 16, 24 flows into one receiver core (Fig. 6); outcast with 1, 8, 16, 24 flows from one sender core (Fig. 7), reported as throughput per sender core.
Question	What happens when more and more flows share one core?
Setup	Incast-N: N flows into receiver CPU2 (senders alternate CPU0/CPU1). Outcast-N: N flows out of sender CPU0 (receivers alternate CPU2/CPU3). N = 1, 2, 4, 8 (the paper goes to 24).
Steps	Run in the same session and random order as 7.3, three runs each.
Measured	Total throughput, and throughput per busy core of the shared CPU (CPU2 for incast, CPU0 for outcast), the paper's metric in Figs. 6 and 7.
Reading the figure	A falling line means each extra flow makes the shared core less efficient.
Run it	Part of <code>sudo python3 code/exp_traffic_patterns.py</code> ; only these: <code>--patterns single,incast,incast-4,incast-8,outcast,outcast-4,outcast-8</code> .
Commands inside	<pre># incast-8: eight server processes all on CPU 2 (ports 5301-5308), # clients alternate: taskset -c 0 / taskset -c 1 # outcast-8: eight clients all on CPU 0, # servers alternate: taskset -c 2 / taskset -c 3</pre>



Flow-count scaling. Left: total throughput. Right: throughput per busy core of the shared CPU (receiver CPU2 for incast, sender CPU0 for outcast).

Pattern	Flows	Total Gbps	Gbps per busy shared core	CPU0 / CPU2 busy %
---------	-------	------------	---------------------------	--------------------

Incast	1	103.3 ± 2.6	133.4 ± 3.8	100 / 77
Incast	2	142.8 ± 1.9	142.8 ± 1.9	66 / 100
Incast	4	128.4 ± 4.8	128.4 ± 4.8	59 / 100
Incast	8	126.6 ± 0.3	126.6 ± 0.3	59 / 100
Outcast	1	103.3 ± 2.6	103.4 ± 2.6	100 / 77
Outcast	2	103.1 ± 0.8	103.1 ± 0.8	100 / 37
Outcast	4	104.1 ± 3.0	104.1 ± 3.0	100 / 36
Outcast	8	103.1 ± 1.8	103.1 ± 1.8	100 / 36

What we see. Incast: per busy receiver core, 133.4 Gbps with 1 flow and 126.6 with 8 (−5 %). Outcast: per busy sender core, 103.4 → 103.1 Gbps (no change).

Why. More flows on one core means more sockets, more wake-ups and context switches, and data of many flows competing for the same cache. The paper measured the cache part with miss counters; we cannot (no PMU), so we see only the end effect on throughput per core.

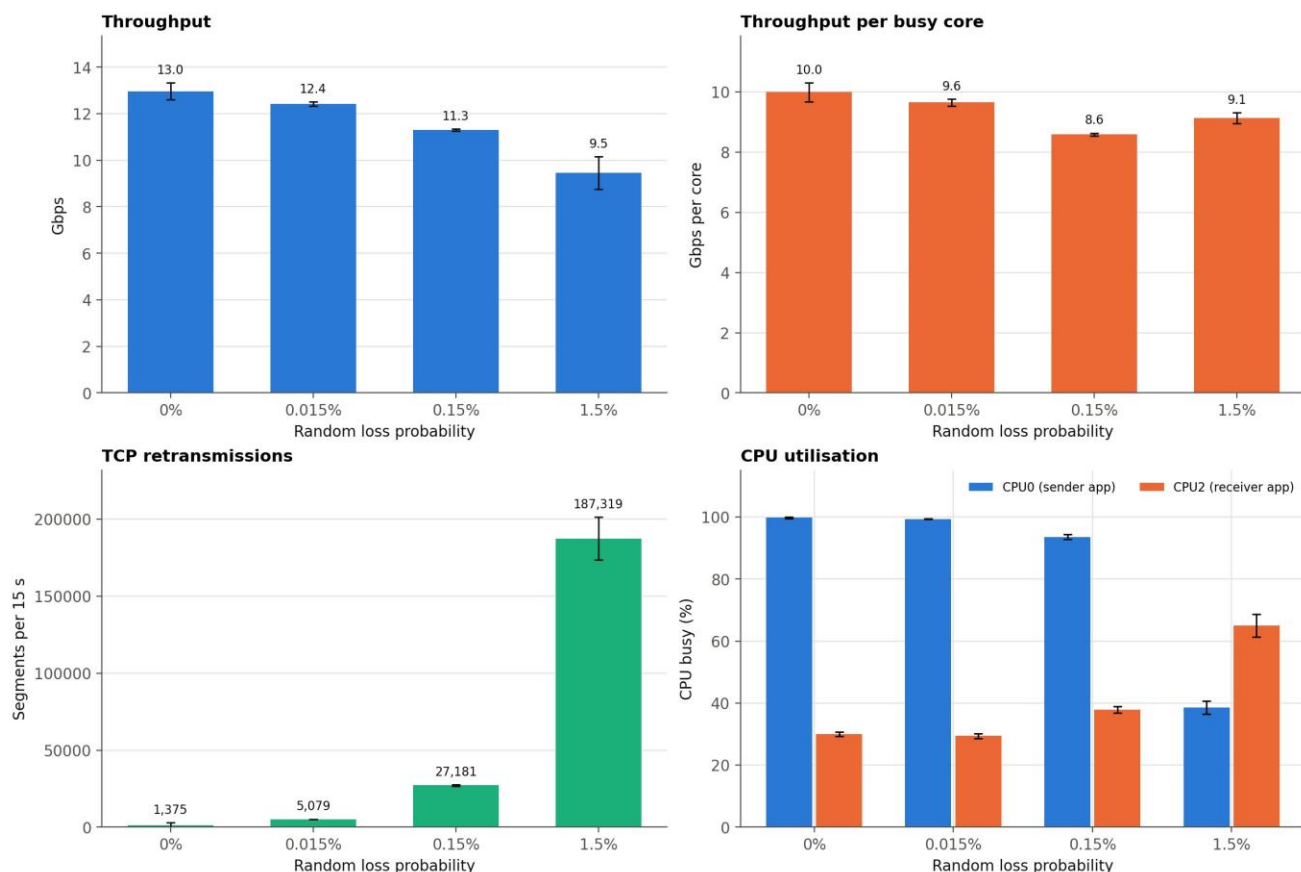
Why outcast is flat here but rises in the paper. In the paper one flow was limited by its receiver, so the sender core had spare capacity that extra flows (to other receiver cores) could use, up to 89 Gbps. In our VM the sender core is already 100 % busy with one flow, so extra flows only divide the same capacity: the total stays at about 103 Gbps from 1 to 8 flows.

7.5 Packet loss (paper §3.6, Fig. 9)

Paper. A switch drops packets at random (0 , $1.5 \cdot 10^{-4}$, $1.5 \cdot 10^{-3}$, $1.5 \cdot 10^{-2}$). At the highest rate throughput-per-core falls about 24 %. Receivers send more duplicate ACKs, senders process them and retransmit, so TCP/IP and netdevice costs rise, more at the sender.

In the paper	Single flow with all optimisations. A programmable switch between the servers drops packets at random with rate 0 , $1.5 \cdot 10^{-4}$, $1.5 \cdot 10^{-3}$ and $1.5 \cdot 10^{-2}$; CPU breakdown at both ends.
Question	How does random packet loss change throughput and CPU efficiency?
Setup	One flow CPU0 → CPU2, CUBIC, TSO/GSO/GRO off, MTU 1500. Packets arriving at the receiver are redirected to an IFB device where <code>netem</code> drops them at random (the paper uses a switch); ACKs are not dropped. Script <code>exp_packet_loss.py</code> .
Steps	For each run: delete and re-create the <code>netem</code> queue with the loss rate (0 , 0.015 , 0.15 or 1.5 %) and a new seed; save <code>nstat</code> and <code>softnet</code> counters; run $3\text{ s} + 15\text{ s}$; save the counters again and <code>netem</code> 's packet and drop counts. Three runs per rate, random order.
Measured	Throughput, retransmissions, measured loss rate (drops ÷ packets), CPU0 and CPU2 busy, throughput per busy core.
Reading the figure	Compare each loss rate with 0 %: what the drops cost in speed and efficiency, and how the work moves between the cores.
Run it	<code>sudo python3 code/exp_packet_loss.py</code> (4 loss rates × 3, ≈ 5 min).
Commands inside	<pre> ip netns exec pl_tx ethtool -K pl_tx0 tso off gso off gro off # and pl_rx0 ip -n pl_rx link add pl_ifb type ifb ; ip -n pl_rx link set pl_ifb up ip netns exec pl_rx tc qdisc add dev pl_rx0 handle ffff: ingress ip netns exec pl_rx tc filter add dev pl_rx0 parent ffff: protocol ip pref 10 matchall action mirrored egress redirect dev pl_ifb ip netns exec pl_rx tc qdisc del dev pl_ifb root # fresh counters every run ip netns exec pl_rx tc qdisc add dev pl_ifb root netem limit 10000 loss random 0.15% seed 1004 ip netns exec pl_tx taskset -c 0 iperf3 -c 10.231.3.2 -p 5301 -C cubic -O 3 -t 15 -J ip netns exec pl_rx tc -s -j qdisc show dev pl_ifb # packets, drops -> measured loss ip netns exec pl_tx nstat -asz ; cat /proc/net/softnet_stat # counters before and after </pre>

Why offloads are off here. With TSO on, a 64 KB skb crosses the veth whole and netem would drop 44 wire packets at once; a switch drops single packets. With TSO, GSO and GRO off every skb is one 1500-byte packet, so the loss probability applies per packet. This makes the 0 % case much slower than the default single flow: it is the "No opt." configuration of 7.1.



Random loss. Throughput, throughput per busy core (CPU0 + CPU2), TCP retransmissions and CPU utilisation.

Loss (set)	Loss (measured)	Throughput (Gbps)	Gbps per busy core	Retrans. / 15 s	CPU0 / CPU2 busy
0 %	0.0000 %	13.0 ± 0.4	10.0 ± 0.3	1,375	100 % / 30 %
0.015 %	0.0150 %	12.4 ± 0.1	9.6 ± 0.1	5,079	99 % / 29 %
0.15 %	0.1505 %	11.3 ± 0.0	8.6 ± 0.0	27,181	94 % / 38 %
1.5 %	1.5015 %	9.5 ± 0.7	9.1 ± 0.2	187,319	39 % / 65 %

What we see. From 0 % to 1.5 % loss, throughput changes by −27 % and throughput per busy core by −9 % (paper: about −24 %). The measured loss matches the configured probability, which validates the method. Retransmissions grow roughly in proportion to the loss rate.

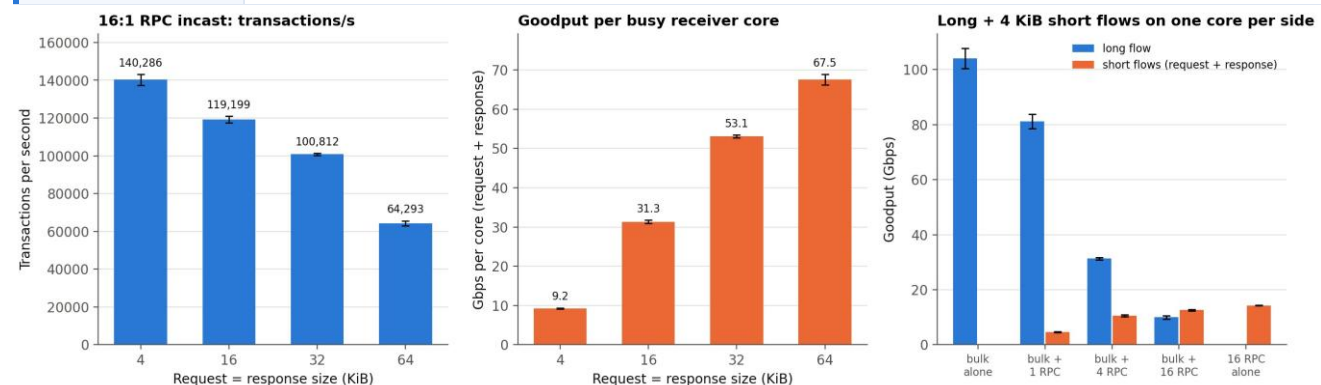
Why. Every lost packet costs duplicate ACKs, a retransmission and a congestion-window reduction (CUBIC cuts its window to 70 % when it detects a loss and then grows it again), so fewer bytes are delivered for the same work. At 1.5 % loss CPU0 is only 39 % busy: the sender is no longer limited by its core but by the congestion window, which is why throughput per busy core falls less than throughput. CPU2 gets busier (30 % → 65 %): a retransmission is usually sent when a SACK arrives, SACKs are processed on CPU2, so with veth the retransmitted packets are also received on CPU2, where TCP merges its out-of-order queue. The paper also finds more ACK work at the receiver (4.9× more time generating ACKs), but in its setup the sender's CPU share grows; here the veth effect moves the extra work to the receiver's core.

Retransmissions at 0 % loss (1,375 on average): netem dropped nothing, so they come from the host. The kernel counters saved with each run (`nstat`) show what happened: the receiver counted outof-order packets (`TCPOFOQueue`) and reported duplicates back to the sender (`TCPDSACKRecv`), and the sender counted reordering (`TCPSTReorder`, `TCPACKReorder`). So most of these retransmissions were *spurious*: the packet was late, not lost. The cause is again the veth. New data is sent partly from CPU0 (the application) and partly from CPU2 (when CPU2 processes an ACK, TCP may send the next packets at once), and each CPU delivers what it sent through its own backlog queue, so the two streams can overtake each other. Real drops in those queues were rare (0 to about 100 packets per run in `/proc/net/softnet_stat`).

7.6 Short flows and mixed workloads (paper §3.7, Figs. 10 and 11)

Paper. (a) 16 applications send ping-pong RPCs (request = response = 4–64 KB) to one receiver core. Throughput-per-core rises with message size; at 4 KB, TCP/IP processing and scheduling (applications blocking and waking for every message) cost more than data copy. (b) One long flow plus 0–16 short 4 KB flows on one core per side: throughput-per-core falls 43 %; the long flow loses 48 % and the short flows 42 % compared with running alone.

In the paper	(a) 16 applications on separate sender cores, each with one long-lived TCP connection doing ping-pong RPCs of 4, 16, 32 or 64 KB with one receiver application on one core (netperf). (b) One core at each end: one long iperf flow plus 0, 1, 4 or 16 short 4 KB RPC flows.
Question	Do short request/response flows cost the CPU differently from long flows, and what happens when both share a core?
Setup	(a) 16 netperf TCP_RR connections (one request outstanding each; request = response = 4, 16, 32 or 64 KiB) from CPUs 0/1 into one receiver core, CPU2 (16:1 incast). (b) One netperf TCP_STREAM long flow (128 KB writes) plus 0, 1, 4 or 16 short 4 KiB RPC flows, all clients on CPU0 and all servers on CPU2; plus 16 RPC flows alone. Script <code>exp_flow_sizes.py</code> .
Steps	netserver, pinned to CPU2, forks one worker per connection; all netperf clients start together and run 20 s; CPU is averaged over seconds 2–19. Three runs per case, random order within each part.
Measured	Transactions per second, application goodput (request + response bytes), long-flow throughput, receiver-core busy %, goodput per busy receiver core.
Reading the figure	Left and middle: how message size changes rate and efficiency. Right: long-flow and shortflow goodput as short flows join the same cores.
Run it	<code>sudo python3 code/exp_flow_sizes.py</code> (9 cases × 3, ≈ 11 min).
Commands inside	<pre> ip netns exec fs_rx taskset -c 2 netserver -D -4 -L 10.231.4.2 -p 12867 # one short flow (16 of these run at once; request = response = 4096 bytes): ip netns exec fs_tx taskset -c 0 netperf -4 -H 10.231.4.2 -p 12867 -l 20 -t TCP_RR -P 0 - v 0 -f x -- -r 4096,4096 # the long flow, 128 KB writes: ip netns exec fs_tx taskset -c 0 netperf -4 -H 10.231.4.2 -p 12867 -l 20 -t TCP_STREAM -P 0 -v 0 -f m -- -m 131072 </pre>



Left: RPC transactions per second for 16 connections into one receiver core. Middle: application goodput (request + response bytes) per busy receiver core. Right: long flow and short flows sharing one core per side.

(a) RPC size

Message	Transactions/s	Goodput req.+resp. (Gbps)	Gbps per busy receiver core	CPU2 busy	CPU0 / CPU1 busy
4KiB	140,286 ± 2,891	9.2 ± 0.2	9.2 ± 0.2	99 %	62 / 62 %
16KiB	119,199 ± 1,755	31.2 ± 0.5	31.3 ± 0.4	100 %	62 / 61 %
32KiB	100,812 ± 655	52.9 ± 0.3	53.1 ± 0.4	100 %	61 / 60 %

64KiB	64,293 ± 1,318	67.4 ± 1.4	67.5 ± 1.4	100 %	57 / 57 %
-------	----------------	------------	------------	-------	-----------

From 4KiB to 64KiB messages the transaction rate falls from 140,286 to 64,293 per second, but the bytes moved per busy receiver core rise from 9.2 to 67.5 Gbps. **Why:** each transaction has a fixed cost (two system calls, a sleep and a wake-up, TCP/IP processing of a small skb, an ACK) plus a cost per byte (the copy). Small messages are dominated by the fixed cost, large ones by the copy, which is the paper's finding.

(b) Mixing a long flow with short flows

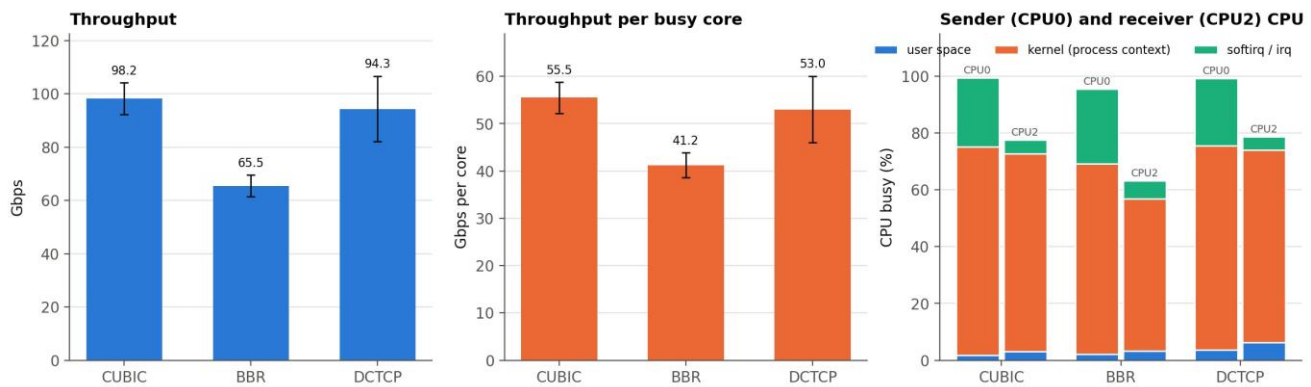
Workload	Long flow (Gbps)	Short-flow transactions/s	Total goodput (Gbps)	CPU0 / CPU2 busy
long flow alone	104.0 ± 3.6	0 ± 0	104.0 ± 3.6	100 % / 77 %
long + 1 short	81.2 ± 2.6	70,519 ± 3,232	85.8 ± 2.8	100 % / 95 %
long + 4 short	31.3 ± 0.5	160,941 ± 6,065	41.9 ± 0.6	100 % / 92 %
long + 16 short	10.0 ± 0.7	192,789 ± 3,638	22.6 ± 0.5	100 % / 98 %
16 short alone	0.0 ± 0.0	217,827 ± 1,357	14.3 ± 0.1	100 % / 99 %

With 16 short flows the long flow drops from 104.0 to 10.0 Gbps (−90 %), and the short flows change by −11 % compared with running alone (paper: −48 % and −42 %). **Why:** the Linux scheduler shares one core fairly between its runnable threads. On CPU2 the long flow's receiver is one of 17 netserver workers, so it gets a small share, while the latency-sensitive RPC workers keep running. The paper's conclusion holds: long and short flows should not share a core.

7.7 Congestion control: CUBIC, BBR, DCTCP (paper §3.10, Fig. 13)

Paper. The three algorithms give almost the same throughput-per-core: the receiver is the bottleneck and congestion control lives at the sender. BBR has more scheduling overhead at the sender because it paces packets (qdisc timers wake the sender repeatedly).

In the paper	Single flow with all optimisations; CUBIC (the default), BBR (paced by the qdisc) and DCTCP; sender and receiver CPU breakdown.
Question	Does the congestion-control algorithm change CPU efficiency?
Setup	One flow CPU0 → CPU2, MTU 1500, fq qdisc on both ends (pacing for BBR; ECN CE mark after 1 ms of queueing for DCTCP), ECN on. CUBIC, BBR, DCTCP. Script <code>exp_congestion_control.py</code> .
Steps	Set the algorithm in both namespaces and with <code>iperf3 -C</code> ; re-create the queues; run 3 s + 15 s; read the live socket twice with <code>ss -ti</code> to prove which algorithm ran (and that DCTCP had ECN); three runs each, random order.
Measured	Throughput, retransmissions, CPU0/CPU2 busy and their split, throughput per busy core, ECN marks.
Reading the figure	Equal bars mean the algorithm does not matter; the CPU panel shows where any extra cost appears.
Run it	<code>sudo modprobe tcp_bbr tcp_dctcp then sudo python3 code/exp_congestion_control.py (3 × 3, ≈ 4 min).</code>
Commands inside	<pre> ip netns exec cc_tx sysctl -w net.ipv4.tcp_ecn=1 # both namespaces ip netns exec cc_tx sysctl -w net.ipv4.tcp_congestion_control=dctcp # both namespaces ip netns exec cc_tx tc qdisc add dev cc_tx0 root fq limit 10000 flow_limit 1000 ce_threshold 1ms ip netns exec cc_tx taskset -c 0 iperf3 -c 10.231.5.2 -p 5301 --cport 41000 -C dctcp -O 3 -t 15 -J ip netns exec cc_tx ss -tinH 'sport = :41000' # live algorithm and DCTCP's ECN state ip netns exec cc_tx tc -s qdisc show dev cc_tx0 # ce_mark = packets marked by fq </pre>



Congestion control. Throughput, throughput per busy core (CPU0 + CPU2), and CPU time split of CPU0 and CPU2.

Algorithm	Throughput (Gbps)	Gbps per busy core	CPU0 / CPU2 busy	Retrans.
CUBIC	98.2 ± 6.0	55.5 ± 3.3	99 % / 78 %	24
BBR	65.5 ± 4.0	41.2 ± 2.6	96 % / 63 %	2,903
DCTCP	94.3 ± 12.3	53.0 ± 7.0	99 % / 79 %	10

Source: the complete congestion-control run of 9 September (same settings: MTU 1500, fq with 1 ms CE threshold, ECN on). BBR and DCTCP need `modprobe`, i.e. real root, so this experiment was not repeated with the new scripts; its CPU numbers are recomputed from its own `mpstat` logs. `sudo bash code/run_all.sh` re-runs it.

What we see. BBR –33 % throughput vs CUBIC; DCTCP –4 % throughput vs CUBIC.

Why. DCTCP ≈ CUBIC: DCTCP slows down only in proportion to ECN marks. A veth never builds a queue longer than 1 ms, so (almost) no packets are marked and DCTCP simply keeps its window open, like CUBIC without loss. **BBR is slower here**, unlike in the paper: BBR paces its packets through fq, which costs timers and extra wake-ups on the sender core (the overhead the paper saw in its sender breakdown). In the paper that extra sender work did not matter because the receiver was the bottleneck; in our VM the *sender* core is the bottleneck, so the same overhead directly lowers throughput. BBR also retransmits more because it does not treat loss as its main signal. Both observations support the paper's point: when the host is the bottleneck, the congestion-control algorithm matters mainly through its CPU cost at the busiest side.

7.8 Where the CPU cycles go (paper Table 1, Figs. 3c/3d)

In the paper	perf sampling on sender and receiver; the functions covering about 95 % of CPU time classified into the 8 categories of Table 1; shown for every configuration (Figs. 3c/d, 5b/c, 6b, 7b, 8b, 9c/d, 10b, 11b, 12b/c, 13b/c).
Question	Which parts of the kernel use the CPU time (the paper's Table 1 categories)?
Setup	Same flow and levels as 7.1. <code>perf record -e cpu-clock -F 999 -C 0,2</code> samples CPU0 and CPU2 about 1000 times per second. Script <code>exp_cpu_profile.py</code> (needs sudo).
Steps	Start the flow; after the 4 s warm-up record 10 s; export with <code>perf script</code> ; map every function name to a category by fixed rules (e.g. <code>__arch_copy_to_user</code> → data copy, <code>tcp_*</code> → tcp/ip, <code>*gro*</code> → netdevice); drop idle samples.
Measured	Fraction of each core's busy time per category; the top 30 functions per core with their category.
Reading the figure	The tallest bars are the biggest costs; comparing levels shows what each optimisation removes.
Run it	<code>sudo python3 code/exp_cpu_profile.py</code> (4 levels, ≈ 3 min; needs <code>linux-tools-\$(uname -r)</code>).
Commands inside	<pre>perf record -e cpu-clock -F 999 -C 0,2 -o no-opt.perf.data -- sleep 10 perf script -i no-opt.perf.data -F comm,cpu,ip,sym,dso # one line per sample # each function name -> one of the 8 categories -> breakdown.csv, top-symbols.csv</pre>

Paper. perf profiles sorted into eight categories show data copy as the largest cost at the receiver (about half of the cycles with all optimisations), and TCP/IP processing dominating without offloads.

Status. `exp_cpu_profile.py` implements the method: for each single-flow level it runs `perf record -e cpu-clock -F 999 -C 0,2` for 10 s of steady state, converts the samples with `perf script`, drops idle samples and sorts every kernel function into the paper's categories with the rules printed in its `configuration.json`. It writes `breakdown.csv` (fractions per category) and `top-symbols.csv` (the top 30 functions per core with their category, so every classification can be checked). `perf` needs real root, so this step runs only with `sudo python3 code/exp_cpu_profile.py` (included in `run_all.sh`); this report is regenerated with the result afterwards.

What mpstat already shows. The CPU panels of Sections 7.1 and 7.7 split each core's time into user space, kernel on behalf of the process (system calls: mainly the data copy and TCP send/receive) and softirq (packet processing). That coarse split already shows the key point of Section 5: without offloads the sender core spends most of its time in softirq, doing receive processing.

Expected result (to be confirmed by the profile): data copy the largest single category once TSO/GSO/GRO are on; TCP/IP and netdevice work dominant with "No opt."; receive-path functions (`ip_rcv`, `tcp_v4_rcv`, `process_backlog`) visible on the *sender* core because of the veth effect.

7.9 Paper §3.8, Impact of DCA (DDIO): not reproducible here

What the paper did. It repeated the single flow of §3.1 with Intel DDIO switched off, so the NIC wrote packets to DRAM instead of the L3 cache. **What it found:** throughput-per-core fell by 19 % and the benefit of aRFS halved, because the receiver now copies every byte from memory instead of cache. The CPU breakdown did not change much.

Why we cannot do it. DDIO is a feature of Intel Xeon processors and of PCIe network cards that write into memory by DMA. Our laptop has an Apple processor, and the veth "cable" has no network card and no DMA: the receiver reads the same memory the sender's copy just wrote. There is no switch to turn off.

What would be needed. A server with an Intel Xeon and a 25–100 Gbps NIC on which DDIO can be switched off (a platform setting on Intel servers), and access to the hardware counters to count L3 misses.

What our data says instead. Our receiver behaves like "perfect DDIO": its copy is cheap, which is one reason the receiver core is not our bottleneck (Sections 5 and 7.1).

7.10 Paper §3.9, Impact of IOMMU: not reproducible here

What the paper did. It repeated the single flow with the IOMMU switched on (it was off in all other experiments). **What it found:** throughput-per-core fell by 26 %; memory allocation/deallocation rose to 30 % of receiver cycles, because the driver must map every new DMA page into the IOMMU's page table and unmap it after the DMA.

Why we cannot do it. The IOMMU only translates addresses used by a *device* doing DMA. In our data path no device does DMA; veth hands over pointers inside one kernel, so there is nothing for an IOMMU to translate, and switching it on would change nothing.

What would be needed. A physical (or emulated) NIC doing DMA. A partial approximation is possible outside this VM: configure the VM in UTM/QEMU with a virtual IOMMU (`virtio-iommu`) and a virtio network card, then send traffic to the Mac host with the IOMMU on and off. That would show part of the mapping cost; it is not the paper's setup, so we list it as future work.

7.11 Other paper measurements not possible here

- **NIC-remote NUMA node (Fig. 4, –20 %):** the VM has one NUMA node.
- **L3 cache-miss rates (Figs. 3e, 4, 6c, 10c):** need hardware counters, which this VM does not expose.
- **NIC receive descriptors (Fig. 3e) and the NAPI-to-copy latency (Fig. 3f):** need a real NIC and custom kernel instrumentation.

8. Discussion: our results against the paper

The table below puts each quantitative claim of the paper that we could test next to our result, says whether the two agree, and gives the reason. The reasons come from our per-CPU measurements (busy time split into user, kernel and softirq on each core), from the kernel counters we saved with each run, and from how the veth driver works. Where the paper explains something with cache-miss counters, we say so, because our VM has no such counters (Section 5).

Paper's finding	Our result	Agreement	Why
-----------------	------------	-----------	-----

Offloads (TSO/GSO/GRO) are the biggest single optimisation (§3.1)	13.5 → 60.9 Gbps per busy core (4.5×)	Agrees	Per-packet work (TCP/IP, queueing, the link hand-over) is paid once per 64 KB instead of once per 1500 B.
Jumbo frames give a further clear gain (§3.1)	−3 % per busy core, i.e. no gain	Differs	In the paper GRO has to merge 1500byte frames coming from the NIC, and jumbo frames give it 6× fewer. A veth passes the 64 KB skb through unsplit, so the MTU stops mattering once TSO is on.
The receiver is the bottleneck (§3.1)	The sender core is the bottleneck (CPU0 100 %, CPU2 77 %)	Differs	veth queues each packet on the backlog of the CPU that sent it, so the receiver's IP/TCP work runs on the sender's core. Our receiver also copies data that the sender core wrote a moment earlier and that is still in cache, not data a NIC wrote to memory.

Paper's finding	Our result	Agreement	Why
About 42 Gbps per core with all optimisations (§3.1)	60.9 Gbps per busy core counting both hosts; 108 Gbps per core on the bottleneck side	Higher	No NIC, DMA, interrupt or cache-miss cost from DMA; the data stays in the processor cache; receive work is spread over two cores.
Large TCP buffers lower throughput per core (§3.1, Fig. 3e)	Flat from 400 KB to 3200 KB (about 107 Gbps); only 100 KB is slower (51.5 Gbps)	Differs	The paper's decline comes from DDIO cache eviction, which needs a NIC writing into the L3 cache; the VM has none. The small-buffer limit ($\text{throughput} \leq \text{buffer} \div \text{RTT}$) is the same in both.
One-to-one: efficiency per core falls as flows are added (§3.2)	Two pairs: 1.57× one flow, −22 % per busy core	Agrees	Independent core pairs still share caches and memory bandwidth, and in our case one hypervisor.
Incast: about −19 % per receiver core at 8 flows (§3.3)	133 → 127 Gbps per busy receiver core (−5 %) at 8 flows	Agrees in direction	More sockets, wake-ups and cache competition on the shared core. Our receiver core does a smaller share of the total work than the paper's, so the effect is smaller.
Outcast: throughput per sender core rises to 89 Gbps at 8 flows (§3.4)	Flat at about 103 Gbps per sender core from 1 to 8 flows	Differs	In the paper the sender core had spare capacity because the receiver was the limit. Our sender core is already 100 % busy with one flow, so extra flows only share it.
All-to-all: efficiency per core falls with many flows (§3.5)	165 Gbps total, −21 % per busy core compared with one flow	Agrees	Four flows over four cores: every core is shared, and each flow gets fewer packets per interval, so GRO merges less.

1.5 % random loss: about –24 % per core (§3.6)	–27 % throughput, –9 % per busy core; measured loss 1.502 %	Agrees in direction	Duplicate ACKs, retransmissions and window reductions cost work per delivered byte. The per-core drop is smaller in our case because the sender core stops being fully busy once the congestion window limits the flow.
Small RPCs are dominated by permessage cost, large ones by the copy (§3.7)	9.2 Gbps per busy receiver core at 4 KB, 67.5 at 64 KB	Agrees	Each transaction pays two system calls, a sleep and wake-up, TCP/IP processing and an ACK, whatever its size.
Long + 16 short flows on one core: long –48 %, short –42 % (§3.7)	Long flow –90 %, short flows –11 %	Agrees in direction	The scheduler shares the receiver core fairly among 17 threads. The single long-flow thread gets one share, the 16 RPC threads together get the rest, so in our setup the long flow loses more and the short flows less than in the paper.
CUBIC, BBR and DCTCP give the same throughput per core (§3.10)	DCTCP –4 % vs CUBIC; BBR –33 % vs CUBIC	Agrees for DCTCP, differs for BBR	BBR's pacing costs timers and wakeups on the sender core. In the paper the receiver is the bottleneck, so this cost is hidden; in our setup the sender core is the bottleneck, so it shows directly.
Paper's finding	Our result	Agreement	Why
DDIO off costs 19 %; IOMMU on costs 26 % (§3.8, §3.9)	Not reproducible	n/a	Both need a physical network card doing DMA; see Section 5.

8.1 One mechanism explains most of the differences

Every "differs" row above has the same root cause: **where the receive processing runs**. With a physical network card and aRFS, the card raises an interrupt on the receiver's core and all receive work (NAPI polling, GRO, IP, TCP and the copy to the application) happens there. The veth driver has no interrupt. Its transmit function calls `netif_rx()`, which puts the packet on the backlog queue of *the CPU that is running at that moment*, and the NET_RX softirq processes it on that same CPU. For data packets that CPU is the sender's core. We can see this directly: with offloads off, CPU0 spends 64 % of its time in softirq although it only runs the sending application. So our sender core does its own work plus most of the receiver's, it saturates first, and anything that adds work at the sender (BBR pacing, retransmissions, more flows out of one core) costs throughput. In the paper the same costs were absorbed by a sender that had spare capacity. Our single-flow script has a fourth level, RFS, which steers receive processing to the receiver's core the way aRFS does in the paper. It needs a global kernel setting and therefore root, and we did not get to run it before this report; it is listed under future work.

8.2 Why our absolute numbers are higher than the paper's

The paper's best single flow reaches about 42 Gbps per core; ours reaches 60.9 Gbps per busy core counting both hosts. Three costs that the paper pays do not exist in our setup: the network card and its driver (descriptor rings, DMA setup, interrupts), the memory traffic caused by DMA (in the paper even one flow has a 49 % L3 miss rate at the receiver), and putting the data on a wire. A veth "transmits" by handing over a pointer, and the receiver copies data that is still in the processor cache. A reproduction like ours can test the direction and the ranking of each effect; the absolute level depends on the hardware and is not expected to match.

8.3 What the reproduction confirms

- The host is the bottleneck once the link is fast enough: one CPU core sits at 100 % in every singleflow run while the link (the veth) has unlimited capacity.
- Without offloads the per-packet costs dominate, and the offloads remove them. This is the paper's Figure 3a in our setting.

- Sharing host resources between flows costs efficiency in every pattern where a core is shared, and even when only caches are shared (one-to-one).
- Packet loss, small messages and mixed workloads each raise the CPU cost per byte, in the direction the paper reports.
- The paper's advice that long and short flows should not share a core, and that the CPU scheduler should know about the network stack, follows from our data as well.

9. Implementation, and what we fixed in our first version

We started this project with an earlier version of the code. Before running new experiments we went through that version line by line against the paper and found that several of the numbers it produced could not be compared with the paper at all. This section records what we built, what was wrong, why it mattered and what replaced it, because the corrections are part of the work and because they explain why the current results differ from our earlier ones.

9.1 What we built

- **One shared module** (`common.py`) that creates and deletes the two namespaces and the veth, sets MTU and offloads with read-back checks, runs iperf3 flows with CPU pinning, aligns the mpstat window to iperf3's own timestamps and computes throughput per busy core for both hosts and for the bottleneck side.
- **One script per paper section** (single flow and buffers; traffic patterns and flow scaling; packet loss; flow sizes; congestion control; CPU profile), each with a fixed random order, three repetitions, a configuration record and a completion marker.
- **An analysis script** that reads only complete result folders, computes mean and standard deviation, draws every figure in this report and writes a summary file. The report and the slides are generated from that file, so no number is typed by hand.
- **Automatic checks:** measured loss rate, live congestion-control algorithm, offload settings, idle baseline.
- **A live demonstration script** (`demo.sh`) that shows the offload effect and incast in about three minutes.

9.2 Errors in the first version and their fixes

The first version of the project (the scripts and results of 9 September) was checked line by line against the paper. These problems were found; all are fixed in the current scripts. We kept a copy of the old version outside the repository.

#	Problem	Why it matters	Fix
1	Only total throughput was reported. The paper's metric, throughput-per-core, was never computed, although mpstat logs were recorded for every run.	All of the paper's conclusions are about CPU cost per byte. Comparing total Gbps of patterns that use different numbers of cores (e.g. "four-flow / single-flow ratio") is not the paper's comparison.	Per-core CPU from mpstat in the measured window; throughput per busy core for both hosts and for the bottleneck side, per run.
2	CPU figures in the loss and congestion-control graphs came from iperf3's <code>cpu_utilization_percent</code> .	That is only the iperf3 process's own time; it misses softirq packet processing (which in veth often runs on the other side's CPU).	mpstat per core, like the paper's sysstat.
3	Packet loss: <code>tc qdisc replace</code> kept netem's counters across runs, so each run's saved drop count was cumulative (a 0 % run showed 485,973 drops).	Per-run loss statistics were wrong. (Recomputing the old counters as differences shows that the <i>configured</i> loss itself had been correct.)	The queue is deleted and recreated per run; the measured loss rate is stored per run.

4	No script created the <code>hsp_tx/hsp_rx</code> namespaces used by five experiments, and the single-flow baseline was run by hand (<code>sender.sh</code> , <code>receiver.sh</code>).	Their settings (MTU, offloads, qdisc) were not recorded, so nobody could rebuild the exact setup.	Every experiment creates, records and deletes its own namespaces.
5	Four almost identical scripts (<code>one_to_one</code> , <code>incast</code> , <code>outcast</code> , <code>all_to_all</code>), six plot scripts, and <code>packet_loss.py</code> importing helpers from <code>one_to_one.py</code> .	Copy-paste code drifts apart; a fix in one copy is missed in the others.	One shared module (<code>common.py</code>), one script per paper section, one analysis script.
6	Patterns ran in separate sessions in a fixed order, and the plots used whichever folder was newest. Two complete all-to-all sessions disagreed (means 144.1 and 173.9 Gbps); only the second was plotted.	Session-to-session drift was mistaken for a pattern effect, and picking the newest folder is an arbitrary selection.	All patterns interleaved in one seeded random order; all repetitions reported with SD.
7	Aborted and partial runs were left in <code>results/</code> (an all-to-all run that stopped after 8 s; two partial congestion-control runs).	A plot script could pick them up.	<code>COMPLETED.txt</code> only after all runs succeed; the analysis ignores anything else. Deleted.
8	Background load was not measured: during the first single-flow run, CPUs 1 and 3 (not used by the test) were up to 71 % busy (26 % on average), and that run was 13 % slower than the other two.	Other programs compete with the experiment and add variation.	Idle baseline recorded before every experiment and reported (Section 6.4).
9	In the flow-size experiment the long flow used netperf's default 16 KB writes and reached 36.6 Gbps, while every other experiment used iperf3's 128 KB writes ($\approx$ 98 Gbps).	The "long flow" of §3.7 was not the same long flow as in the other sections.	netperf <code>-m 131072</code> (128 KB writes).
10	Paper items that are really part of §3.1–3.5 were missing: the step-by-step optimisations (Fig. 3a), the TCP buffer study (Fig. 3e), flow-count scaling (Figs. 6/7) and the CPU breakdown of Table 1.	These are core results of the paper.	Added: <code>exp_single_flow.py</code> , scaling patterns, <code>exp_cpu_profile.py</code> .
11	Leftover files: <code>congestion_control.before*.py</code> , <code>run_congestion_control.py</code> (not needed, because root may select any loaded algorithm), root-owned <code>__pycache__</code> and result folders, old notes describing namespaces that no longer exist.	Clutter; files could not be edited without sudo.	Removed; results are returned to the user after sudo runs.

What was already right, and kept

CPU pinning with taskset; one process pair per flow; 3 s warm-up + 15 s measurement; checking the live congestion-control algorithm with `ss` and the corrected DCTCP/ECN check; netem on an IFB at the receiver with offloads off for per-packet loss; seeded random order in the loss and flowsize experiments; receiver-side throughput; honest notes in the graphs.

10. Non-functional evaluation

Apart from the functional question (does the reproduction show the paper's effects?) we also looked at the qualities of the measurement setup itself. The table gives, for each parameter, what we measured, how, and the result.

Parameter	What we measured	How	Result
Performance	Throughput (Gbps) and RPC transactions per second	iperf3 / netperf, 3 s warm-up discarded, 15 s (20 s for netperf) measured	108 Gbps with one flow; 165 Gbps with four flows on four cores; up to 140,286 transactions/s at 4 KB

Efficiency	Gbps per busy CPU core (the paper's metric)	mpstat on every core every second, averaged over the measured window; busy = 100 – idle	13.5 → 60.9 Gbps per busy core with offloads (4.5×)
Scalability	More cores (one pair → two pairs) and more flows on one core (1 → 8)	Traffic-pattern and flow-count runs in one random sequence	Two pairs = 1.57× one flow; 8 flows into one receiver core: –5 % per busy core
Parameter	What we measured	How	Result
Reliability / robustness	Behaviour under random packet loss of 0 to 1.5 %	netem on an IFB device at the receiver; drop counters read after every run	–27 % throughput at 1.5 %; measured loss 1.502 %; all data delivered
Repeatability	Spread between the three runs of every condition	Three runs per condition in seeded random order; fresh namespaces for each experiment	Median spread 1.4 % of the mean; largest 7 % (the 1.5 % loss case, where the loss itself is random)
Measurement validity	Background load; settings really in effect	5 s idle baseline before every experiment; ethtool, ss and netem read-back; unused CPUs checked	Idle load 1–13 % per CPU; two disturbed experiments detected and repeated
Reproducibility	Can someone else rerun it and get the same graphs?	One script per experiment; every setting recorded in configuration.json; COMPLETED marker; the analysis reads only complete runs	sudo bash code/run_all.sh rebuilds everything in about 70 minutes
Resource use	Time and disk for a full run	Measured on the 4-vCPU VM	About 45 minutes of measurements; 102 runs; under 10 MB of raw data

10.1 Repeatability

Every condition was run three times. The runs of one experiment are interleaved in a random order fixed by a seed, so a slow change in background load cannot favour one condition over another. The standard deviation divided by the mean is 1.4 % in the median over all conditions. The largest value, 7 %, belongs to the 1.5 % loss case, where randomness is part of the experiment. With three runs we treat a difference smaller than about two standard deviations as "no clear difference" and say so in the text.

10.2 Validity checks built into the scripts

- **Offloads:** after every `ethtool -K` the settings are read back with `ethtool -k`, and the run aborts if they differ from what was requested.
- **Congestion control:** the algorithm is read from the live data socket with `ss -ti` twice during each run, identified by its fixed source port. For DCTCP the script also checks that ECN was negotiated.
- **Packet loss:** the netem queue is deleted and re-created before each run, and its packet and drop counters give the measured loss rate, which is stored next to the configured one.
- **Background load:** a 5-second mpstat baseline is taken before each experiment, and the busy time of the CPUs an experiment does not use is visible in every result, so a disturbed run can be recognised. This is how we caught the two experiments that ran while a second editor window was open, and repeated them (Section 11).
- **Completeness:** a result folder gets `COMPLETED.txt` only when every run succeeded and parsed. The analysis ignores folders without it, so an aborted run cannot appear in a graph.

11. Challenges faced

Challenge	What it meant	What we did
-----------	---------------	-------------

No 100 Gbps hardware, and only one laptop	The assignment expected two machines. A laptop network of about 1 Gbps would be the bottleneck, and the CPU effects would be invisible (Section 4).	Two network namespaces joined by a veth pair inside one VM. The veth has no line rate, so the CPU is the bottleneck, as in the paper.
A virtual machine has no DDIO, IOMMU, NUMA or hardware counters	Two of the paper's ten sections and all of its cache-miss measurements are impossible here.	Explained each one and the hardware it needs (Section 5); reproduced the other eight sections; marked every cache-level explanation as the paper's.
The veth does receive work on the sender's CPU	Our bottleneck moved from the receiver (paper) to the sender, which changes several results.	Measured it (64 % softirq on CPU0 with offloads off), explained every affected result with it (Section 8), and scripted RFS as the remedy for a later run.
Other programs disturbed measurements	A second editor window opened in the VM during our first full run pushed the unused CPUs to 52 % busy and lowered throughput.	Added an idle baseline before every experiment, random run order and a per-CPU view of the unused cores; discarded and repeated the two disturbed experiments.
Misleading numbers	Cumulative netem counters, iperf3's process-only CPU figure, and retransmissions at 0 % loss all looked like results but were artefacts.	Fresh queue per run; per-core mpstat instead of iperf3's figure; kernel counters (<code>nstat</code>) showed that the 0 %-loss retransmissions came from reordering between the two CPUs' backlog queues, not from loss.
Only four vCPUs	The paper uses up to 24 cores per host and 576 flows.	Two CPUs per host; incast and outcast scaled to 8 flows on one core; one-to-one and all-toall with 2×2 cores.
Steps that need root and kernel modules	Loading the BBR and DCTCP modules, the global RFS table and perf need root. We ran the main experiments as an unprivileged user inside a user namespace, so a script error could not damage the VM.	The root-only steps are scripted separately and run with <code>sudo</code> . The congestion-control results come from the complete root run of 9 September (Section 7.7); the RFS level and the perf profile are still to be run (Section 12.2).
Keeping the mistakes of our first version from coming back	Hand-made setups and copy-pasted scripts had drifted apart.	One shared module, one script per section, every setting recorded, and checks that abort a run when a setting is not in effect (Section 9).

12. Conclusion and future work

12.1 Conclusion

We reproduced the measurement study of Cai et al. on one laptop. The paper's two servers became two network namespaces joined by a veth pair inside a four-CPU virtual machine, and we measured the paper's metric, throughput per busy CPU core, from per-CPU load logs. Eight of the paper's ten experiment sections were reproduced, with 34 conditions and 102 runs. The results confirm the paper's qualitative findings: at high speed the host CPU is the bottleneck; segmentation and receive offloads are the optimisation that matters most (4.5× more work per busy core); flows that share cores or caches lose efficiency; and packet loss, small messages and mixed workloads all raise the CPU cost per byte. We could also show, with direct measurements, why two results differ in a virtual machine: the veth driver does the receiver's packet processing on the sender's CPU, so the sender core becomes the bottleneck, which makes jumbo frames irrelevant and makes BBR's pacing cost visible. The paper's lesson, that host resources (cores and caches) must be managed as carefully as network resources, holds in our setting too: how the cores are used matters more than the speed of the link.

The programming side of the project taught us as much as the measurements. Our first version produced numbers that looked like results but either were not the paper's metric or were spoiled by cumulative counters and background load. The current version records every setting, checks that each setting is in effect, repeats every condition in random order and reports the spread, so every number in this report can be traced back to a raw run and re-created with one command.

12.2 Future work

- **Run the two root-only steps.** The RFS level of the single-flow experiment (moving receive processing to the receiver's core, as aRFS does in the paper) and the perf CPU breakdown (Table 1) are implemented and only need `sudo`. They would show how much of the sender-side bottleneck RFS removes, and give the per-category breakdown of Figure 3c/3d.
- **A virtual network card.** Configuring the VM with a virtio network device and a virtual IOMMU, and sending traffic to the laptop's host operating system, would add DMA-like behaviour and allow a partial version of the IOMMU experiment (§3.9).
- **Real hardware.** Two machines with 25–100 Gbps network cards and a direct cable would allow the DDIO experiment, cache-miss counters, NUMA effects and the paper's absolute numbers.
- **More cores and flows.** A VM with 8–16 vCPUs would take one-to-one and all-to-all closer to the paper's 24×24 flows and show the GRO-merging effect at high flow counts.
- **More repetitions** (five or ten per condition) would make the small differences, for example jumbo frames against TSO only, more certain.

References

1. Q. Cai, S. Chaudhary, M. Vuppalapati, J. Hwang and R. Agarwal. Understanding Host Network Stack Overheads. *Proceedings of ACM SIGCOMM 2021*, pp. 65–77. doi:10.1145/3452296.3472888.
2. The Linux kernel documentation. *Scaling in the Linux Networking Stack* (RSS, RPS, RFS, aRFS, XPS). Documentation/networking/scaling.rst.
3. The Linux kernel documentation. *Segmentation Offloads* (TSO, GSO, GRO). Documentation/ networking/segmentation-offloads.rst.
4. S. Ha, I. Rhee and L. Xu. CUBIC: A New TCP-Friendly High-Speed TCP Variant. *ACM SIGOPS Operating Systems Review*, 42(5), 2008.
5. N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh and V. Jacobson. BBR: Congestion-Based Congestion Control. *ACM Queue*, 14(5), 2016.
6. M. Alizadeh, A. Greenberg, D. A. Maltz, J. Padhye, P. Patel, B. Prabhakar, S. Sengupta and M. Sridharan. Data Center TCP (DCTCP). *Proceedings of ACM SIGCOMM 2010*.
7. Intel Corporation. *Intel Data Direct I/O Technology (Intel DDIO): A Primer*, 2012.
8. iperf3 (ESnet), <https://github.com/esnet/iperf>; netperf (Hewlett Packard), <https://github.com/HewlettPackard/netperf>; sysstat (mpstat); Linux perf; iproute2 (ip, tc, ss, nstat); man pages veth(4), ip-netns(8), tc-netem(8), ethtool(8), taskset(1).

Appendix A. How to run everything

```
cd ~/host-stack-paper
sudo apt install iperf3 netperf sysstat ethtool iproute2 linux-tools-$(uname -r) weasyprint #
close VS Code, the browser and Docker first (they compete for the 4 vCPUs) sudo bash
code/run_all.sh # all experiments, about 70 minutes python3 code/analyze.py #
graphs and tables -> analysis/ python3 code/make_final_report.py # this report ->
report/Final_Report.pdf python3 code/make_report.py # the supplement -> report/
```

A single experiment can be repeated on its own, with more repetitions, for example `sudo python3 code/ exp_traffic_patterns.py --reps 5` or `sudo python3 code/exp_single_flow.py --levels no-opt,tso-gro -no-buffers`. Each script prints its results folder; the raw data of every run stays there (iperf3 JSON, mpstat log, server logs, settings).

Script	Paper	Runs	Time
exp_single_flow.py	§3.1, Fig. 3a/b/e	4 levels × 3 + 6 buffers × 3	≈ 12 min
exp_traffic_patterns.py	§3.2–3.5, Figs. 5–8	9 patterns × 3	≈ 11 min
exp_packet_loss.py	§3.6, Fig. 9	4 loss rates × 3	≈ 5 min
exp_flow_sizes.py	§3.7, Figs. 10–11	9 cases × 3	≈ 11 min
exp_congestion_control.py	§3.10, Fig. 13	3 algorithms × 3	≈ 4 min
exp_cpu_profile.py	Table 1, Fig. 3c/d	4 levels	≈ 3 min

Appendix B. Repository layout and data

Path	Contents
code/common.py	Namespaces and veth, running iperf3 flows, mpstat parsing, throughput-per-core
code/exp_*.py	One script per paper section (see Appendix A)
code/analyze.py	Graphs (analysis/fig_*.png), tables (analysis/*.csv) and analysis/summary.json
code/make_final_report.py, code/make_report.py	This report and the supplement
code/run_all.sh, record_environment.sh	Run everything; record the machine's facts in notes/environment.txt
results/<experiment>- <date>/	configuration.json (all settings), runs.csv (one line per run), per run: iperf3 JSON (*flow- PORT.json), mpstat log (*-cpu.txt), CPU window (*-meta.json), server logs; loss: netem/nstat/softnet snapshots; congestion control: ss snapshots and queue stats
analysis/	Figures and summary tables used in this report

Data used in this report

Experiment	Result folder
single flow	single-flow-20260910-233639
traffic patterns	traffic-patterns-20260910-234547
packet loss	packet-loss-20260910-232216
flow sizes	flow-sizes-20260910-232623

Appendix C. Environment record

Recorded by `code/record_environment.sh` on the machine that produced the results.

Recorded: 2026-09-10T23:03:52+00:00

OPERATING SYSTEM

Linux ubuntu 7.0.0-31-generic #31-Ubuntu SMP PREEMPT_DYNAMIC Sat Aug 1 03:33:01 UTC 2026 aarch64
GNU/Linux
PRETTY_NAME="Ubuntu 26.04 LTS"
VERSION="26.04 (Resolute Raccoon)"

VIRTUALISATION

qemu

CPU

Architecture: aarch64
CPU(s): 4
Vendor ID: Apple
Model name: -
Thread(s) per core: 1
Core(s) per socket: 4
Socket(s): 1
NUMA node(s): 1
NUMA node0 CPU(s): 0-3

MEMORY

	total	used	free	shared	buff/cache	available
Mem:	7.2Gi	5.0Gi	330Mi	268Mi	2.3Gi	2.2Gi
Swap:	4.0Gi	325Mi	3.7Gi			

PERFORMANCE COUNTERS

breakpoint
kprobe
software
tracepoint
uprobe
(no cpu/armv8 PMU entry = no hardware cache-miss counters in this VM)

TCP SETTINGS

net.ipv4.tcp_available_congestion_control = reno cubic
net.ipv4.tcp_congestion_control = cubic
net.ipv4.tcp_rmem = 4096131072 33554432
net.ipv4.tcp_wmem = 409616384 4194304
net.core.rmem_max = 4194304
net.core.wmem_max = 4194304
net.core.netdev_max_backlog = 1000
net.core.default_qdisc = fq_codel

TOOLS

iperf3 /usr/bin/iperf3
netperf /usr/bin/netperf
mpstat /usr/bin/mpstat
perf /usr/bin/perf
ethtool /usr/sbin/ethtool
tc /usr/sbin/tc
iperf 3.20 (cJSON 1.7.15)
Netperf version 2.7.1
sysstat version 12.7.7

Appendix D. Glossary

ACK	TCP acknowledgement telling the sender which bytes arrived.
aRFS / RFS	(Accelerated) Receive Flow Steering: process received packets on the core where the application runs (NIC / software).
BBR	Google's congestion control; models bandwidth and RTT and paces packets.
BDP CUBIC	Bandwidth-delay product: data in flight needed to fill the path.
cwnd	Linux's default congestion control; loss-based, cubic window growth.
DCA / DDIO	Congestion window: how much unacknowledged data TCP may have in flight.
DCTCP	Direct Cache Access / Intel Data Direct I/O: NIC DMA into the L3 cache. Datacenter TCP: reduces its window in proportion to the fraction of ECN-marked packets.
DMA	Direct Memory Access: a device reads or writes memory without the CPU.
ECN / CE	Explicit Congestion Notification: a queue marks packets "Congestion Experienced" instead of dropping them.
fq	Fair-queue qdisc; per-flow queues and pacing; can mark CE above a delay threshold.
GRO / LRO	Generic / Large Receive Offload: merge received packets of a flow into large skbs (software / NIC). Generic / TCP Segmentation Offload: split large skbs into MTU packets late (software / NIC).
GSO / TSO	Intermediate Functional Block: virtual device that lets a qdisc act on incoming traffic.
IFB	Translates and protects device DMA addresses.
IOMMU	Bulk TCP throughput tool (one long flow per client).
iperf3 IRQ	Hardware interrupt request.
mpstat	sysstat tool printing per-CPU utilisation (user, system, softirq, idle...).
MTU NAPI	Maximum Transmission Unit: largest packet on the link (1500 or 9000 bytes).
netem	Linux receive API: after an interrupt the driver polls many packets at once. qdisc that emulates delay, loss, reordering.
netns	Network namespace: an isolated network stack.
netperf	Benchmark tool; TCP_RR = request/response, TCP_STREAM = bulk.
NUMA	Non-Uniform Memory Access: memory and devices are closer to some sockets than others.
perf	Linux profiler; samples which function each CPU is running.
PMU	Performance Monitoring Unit: hardware counters (cycles, cache misses).
qdisc	Queueing discipline: the kernel's per-interface packet scheduler.
RSS / RPS	Receive Side Scaling / Receive Packet Steering: spread flows over cores by hash (NIC / software). Round-trip time.
RTT skb	sk_buff, the kernel's packet descriptor; points to the data.
softirq ss	Deferred kernel work, e.g. NET_RX packet processing.
taskset	Socket statistics tool; <code>ss -ti</code> shows TCP internals of live sockets.
veth	Pins a program to chosen CPUs. Virtual Ethernet pair: two interfaces joined back-to-back.